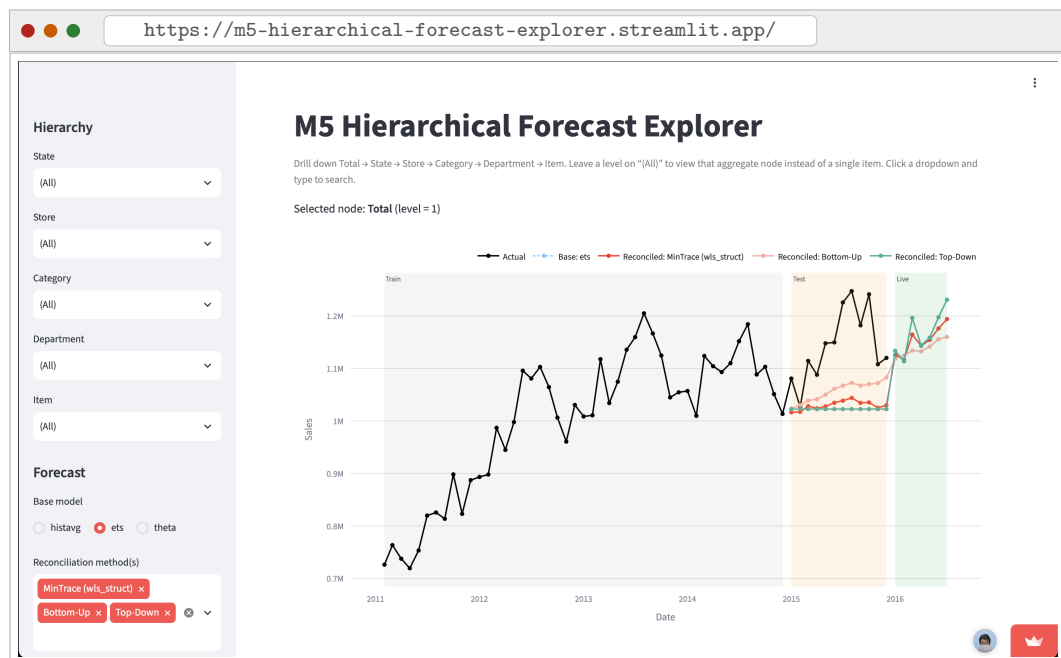


M5 Hierarchical Forecast Explorer

Interactive dashboard for hierarchical sales forecasts



Author: Hemanth Jadiswami Prabhakaran
Matriculation No.: 7026000

Course of Studies: Business Intelligence and Data Analytics

First examiner: Prof. Dr. Joachim Schwarz
Second examiner: Prof. Dr.-Ing. Armando Walter Colombo

Submission date: September 28, 2026

University of Applied Sciences Emden/Leer · Faculty of Technology · Mechanical
Engineering Department
Constantiaplatz 4 · 26723 Emden · <http://www.hs-emden-leer.de>

Contents

List of figures	v
List of tables	vii
Acronyms	ix
1 Introduction and Main Function	1
1.1 Main Function	1
1.2 Key Terms in Plain Words	2
1.3 Primary Capabilities	3
1.4 System Overview	3
1.5 The Dashboard at a Glance	4
1.6 Target Users	5
1.7 Structure of this Manual	5
2 Installation	7
2.1 Option A: Cloud Access (Recommended)	8
2.1.1 Requirements	8
2.1.2 Opening the Dashboard	8
2.1.3 Waking the App Up	8
2.2 Option B: Local Installation	9
2.2.1 Requirements	9
2.2.2 Step 1 – Download the Repository	10
2.2.3 Step 2 – Create a Virtual Environment	10
2.2.4 Step 3 – Install the Packages	10
2.2.5 Step 4 – Check the Result Files	11
2.2.6 Step 5 – Start the Dashboard	11
2.3 Verifying the Installation	13
3 First Steps	15
3.1 Step 1 – Open the Dashboard	15
3.2 Step 2 – Read the Total Chart	15
3.3 Step 3 – Drill Down to a State and a Store	16
3.4 Step 4 – Continue to a Single Item	17
3.5 Step 5 – Switch the Base Model	18
3.6 Step 6 – Add a Second Reconciliation Method	19
3.7 Step 7 – Explore the Chart	20
3.8 Step 8 – Reset to the Total	20

3.9	Summary	20
4	User Interface	23
4.1	Sketch of the User Interface	23
4.2	Layout Principles	24
4.3	The Sidebar	25
4.3.1	Hierarchy Section	25
4.3.2	Forecast Section	26
4.3.3	Collapsing the Sidebar	26
4.4	The Main Area	26
4.4.1	Title, Instructions and Node Line	26
4.4.2	The Forecast Chart	27
4.4.3	Chart Toolbar	28
4.5	Page Menu and Theme	28
4.6	Messages	29
5	Functions	31
5.1	The Sales Hierarchy	31
5.2	Navigating with the Cascading Dropdowns	32
5.3	Searching in a Dropdown	34
5.4	Choosing the Base Model	35
5.5	Choosing Reconciliation Methods	36
5.6	Exploring the Chart	37
5.7	Light and Dark Mode	37
5.8	Typical Analysis Tasks	38
6	Specifications	39
6.1	Overview	39
6.2	Software and Runtime	39
6.3	Client Requirements	40
6.4	Data	41
6.4.1	Source	41
6.4.2	Result Files	41
6.5	Time Windows	42
6.6	Models and Methods	43
6.7	Forecast Accuracy (Reference)	43
6.8	Performance	44
6.9	Limitations	44
7	Maintenance	45
7.1	Maintenance Cycle	45
7.2	Routine Tasks	46
7.3	Managing the Online App	46
7.4	Updating Packages	47
7.5	Changing Settings	48

7.6	Verification Checklist	48
7.7	Regenerating the Result Files	49
7.8	Backup and Rollback	49
7.9	Resetting a Local Installation	50
8	Troubleshooting	51
8.1	The Page Does Not Load	51
8.1.1	“This app has gone to sleep”	51
8.1.2	The Page Stays Blank or Shows “Oh no”	52
8.1.3	<code>streamlit: command not found</code>	52
8.1.4	<code>ModuleNotFoundError</code>	52
8.1.5	Port 8501 Already in Use	53
8.2	Error and Warning Messages	53
8.2.1	“Couldn’t find one of the required parquet files”	53
8.2.2	“No data found for ...”	54
8.2.3	“No column found for ...”	54
8.3	The Chart Looks Wrong	54
8.3.1	No Reconciled Line	54
8.3.2	A Reconciled Line Hides Another Line	55
8.3.3	Colours Hard to Read	55
8.3.4	A Dropdown Offers Only “(All)”	55
8.3.5	Forecast Looks Flat, or Is Zero	56
8.4	General Procedure	56
9	Help	57
9.1	Frequently Asked Questions	57
9.2	Glossary	58
9.3	Quick Reference	59
9.4	Reporting a Problem	59
9.5	Further Information	60
	Bibliography	61

List of Figures

1.1	System overview: the grey part was done once for the thesis; the blue part is what this manual covers.	4
1.2	The dashboard directly after opening, showing the grand total	4
2.1	Choosing between cloud access and local installation	7
2.2	The dashboard opened from Streamlit Community Cloud	8
2.3	The page shown when the app is asleep	9
2.4	Successful installation of the five packages	11
2.5	Terminal output after starting the dashboard locally	12
2.6	The local dashboard at <code>http://localhost:8501</code>	12
3.1	The eight steps of the first session	15
3.2	The grand total with the default settings	16
3.3	Store CA_1 selected; category, department and item are still (A11) . .	17
3.4	A single item, FOODS_3_090 in store CA_1	18
3.5	The same node with the base model theta	19
3.6	All three reconciliation methods compared for one node	20
4.1	Sketch of the user interface with numbered elements	23
4.2	The real user interface for store CA_1 with base model ets	24
4.3	Sidebar: hierarchy section (left) and forecast section (right)	25
4.4	Anatomy of the forecast chart (illustrative data)	27
4.5	The chart toolbar	28
4.6	The same chart in light mode (left) and dark mode (right)	29
5.1	The six-level sales hierarchy with the example path used in this manual	31
5.2	How the five dropdowns determine the node that is shown	33
5.3	With State on (A11), all lower dropdowns are locked to (A11)	34
5.4	Typing 090 in the Item dropdown	35
5.5	Hover information (left) and the test period after zooming in (right) .	37
6.1	Runtime layers of the dashboard	40
6.2	The three time windows on a monthly scale	42
7.1	Maintenance and redeployment cycle	45
7.2	The Manage app panel with logs and app menu	47
8.1	Finding the right section from the symptom	51

List of Figures

8.2	Error message when a result file is missing	54
-----	---	----

List of Tables

1.1	What the dashboard can and cannot do	3
1.2	Chapters of this manual and who needs them	5
2.1	Requirements for a local installation	9
2.2	Installation check	13
4.1	Elements of the user interface (numbers as in Figure 4.1)	24
4.2	Hierarchy dropdowns	26
4.3	Lines and regions of the chart (letters as in Figure 4.4)	27
4.4	Chart toolbar buttons	28
4.5	Messages the dashboard can show	29
5.1	Levels of the hierarchy	32
5.2	The three base models	36
5.3	The three reconciliation methods	36
5.4	Chart interactions	37
5.5	Typical tasks and how to do them	38
6.1	Key facts	39
6.2	Software components (pinned versions from <code>Code/app/requirements.txt</code>)	40
6.3	Result files read by the dashboard	41
6.4	Time windows	42
6.5	Forecasting settings	43
6.6	Average Symmetric Mean Absolute Percentage Error (sMAPE) in % over the test year, by level (ets and theta)	43
6.7	Typical response times	44
7.1	Routine maintenance tasks	46
7.2	Actions in the Streamlit Community Cloud workspace	46
7.3	Deployment settings of the online app	47

Acronyms

GUI Graphical User Interface

M5 Fifth Makridakis Forecasting Competition

PNG Portable Network Graphics

RAM Random Access Memory

sMAPE Symmetric Mean Absolute Percentage Error

URL Uniform Resource Locator

1 Introduction and Main Function

The *M5 Hierarchical Forecast Explorer* is a web dashboard for looking at monthly sales forecasts of Walmart products at any level of detail – from the grand total of all stores down to a single product in a single store. It was built as the application part of the master’s thesis *Hierarchical Time Series Forecasting (A Business - Focused Evaluation of Foundational Reconciliation Methods in Retail Demand)* and shows the thesis results in a form that can be explored without any programming.

This manual explains how to open or install the dashboard, how to use every control on the screen, and how to keep it running. It is written for users of the dashboard. No knowledge of the program code, of Python or of the forecasting methods is needed to follow Chapters 2 to 5. Chapters 7 and 8 address the person who looks after the installation and assume only that they can use a terminal.

1.1 Main Function

The dashboard answers one question:

“For any part of the Walmart sales hierarchy, how did the forecasts compare with what was actually sold – and how do the different forecasting and reconciliation methods differ from each other?”

To answer it, the user picks a point in the hierarchy in the sidebar on the left, and the main area shows a single interactive chart for that point containing

- the **actual** monthly sales,
- one **base forecast**, chosen from three forecasting models, and
- one or more **reconciled forecasts**, chosen from three reconciliation methods.

Everything else in the dashboard – the dropdowns, the model selector, the shaded time regions, the zoom and export tools – serves this one comparison. The dashboard is *read-only*: it does not accept uploads, does not compute new forecasts and does not change any data. All numbers it shows were calculated beforehand and are loaded from four result files (Section 6.4).

1.2 Key Terms in Plain Words

The following terms appear on the screen and throughout this manual. They are explained here only as far as needed to use the dashboard; the thesis contains the full background, and [HA21] offers a readable general introduction to forecasting.

Hierarchy

Walmart’s sales are organised in six levels: **Total** → **State** → **Store** → **Category** → **Department** → **Item**. Each level is the sum of the level below it. Section 5.1 shows the complete structure.

Node

One single point in the hierarchy, for example “all sales of store **CA_1**”. The chart always shows exactly one node.

Actual sales

The number of units that were really sold per month, taken from the Fifth Makridakis Forecasting Competition (M5) data set [MSA22; Kag20].

Base forecast

A forecast made for each node on its own, by one of three models: **histavg** (the historical average), **ets** (exponential smoothing) or **theta** (the Theta method). Base forecasts of different levels usually do not add up – the forecasts for the three states do not sum to the forecast for the total.

Reconciled forecast

A base forecast that has been adjusted so that all levels add up again. The dashboard offers three methods: **Bottom-Up**, **Top-Down** and **MinTrace (wls_struct)** [WAH19].

Train / Test / Live

Three time windows, shaded in the chart. Models learned from the *train* months, were checked against real sales in the *test* months, and look ahead into the *live* months (Section 6.5).

1.3 Primary Capabilities

Table 1.1: What the dashboard can and cannot do

Capability	Details
Navigate the hierarchy	30,354 nodes on six levels, reached with five linked drop-downs (Section 5.2)
Search inside a dropdown	Type part of a name, e.g. 090, to jump to an item (Section 5.3)
Choose the base model	<code>histavg</code> , <code>ets</code> or <code>theta</code> (Section 5.4)
Compare reconciliation methods	Any combination of the three methods in the same chart (Section 5.5)
Explore the chart	Hover values, zoom, pan, hide lines, save as Portable Network Graphics (PNG) (Section 5.6)
Light and dark mode	The chart follows the dashboard's theme automatically (Section 5.7)
<i>Not included (by design)</i>	
Uploading own data	The dashboard only shows the thesis results
Re-training models	Forecasts are pre-computed; see Section 7.7
Downloading tables	Only the chart image can be saved

1.4 System Overview

Figure 1.1 shows the parts involved when the dashboard is used. The forecasts were produced once, offline, by the thesis' forecasting pipeline and stored as four result files in the project's GitHub repository together with the dashboard program. From there, the dashboard can be run in two ways:

- A) **Online** on Streamlit Community Cloud [Sno26a]. Anybody with the link can use it in a browser; nothing has to be installed. This is the normal way.
- B) **Locally** on your own computer, for offline use or for checking and changing the installation.

Both ways show exactly the same dashboard with exactly the same numbers.

1 Introduction and Main Function

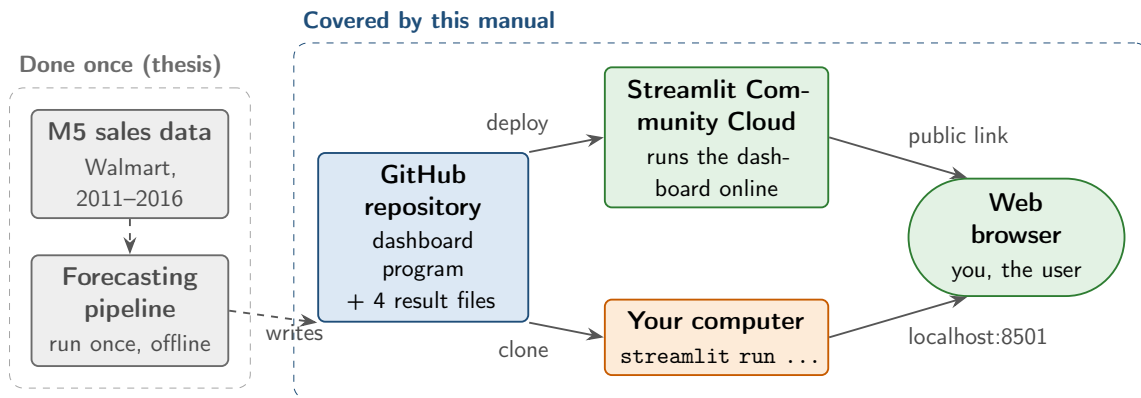


Figure 1.1: System overview: the grey part was done once for the thesis; the blue part is what this manual covers.

Where is the dashboard?

Online: <https://m5-hierarchical-forecast-explorer.streamlit.app/>
Source code and result files: https://github.com/Hemanth-Jp/MBIDA_Master_Thesis

1.5 The Dashboard at a Glance

Figure 1.2 shows the dashboard right after opening. The sidebar on the left holds all controls, the main area on the right holds the chart. Chapter 4 explains every element in detail.

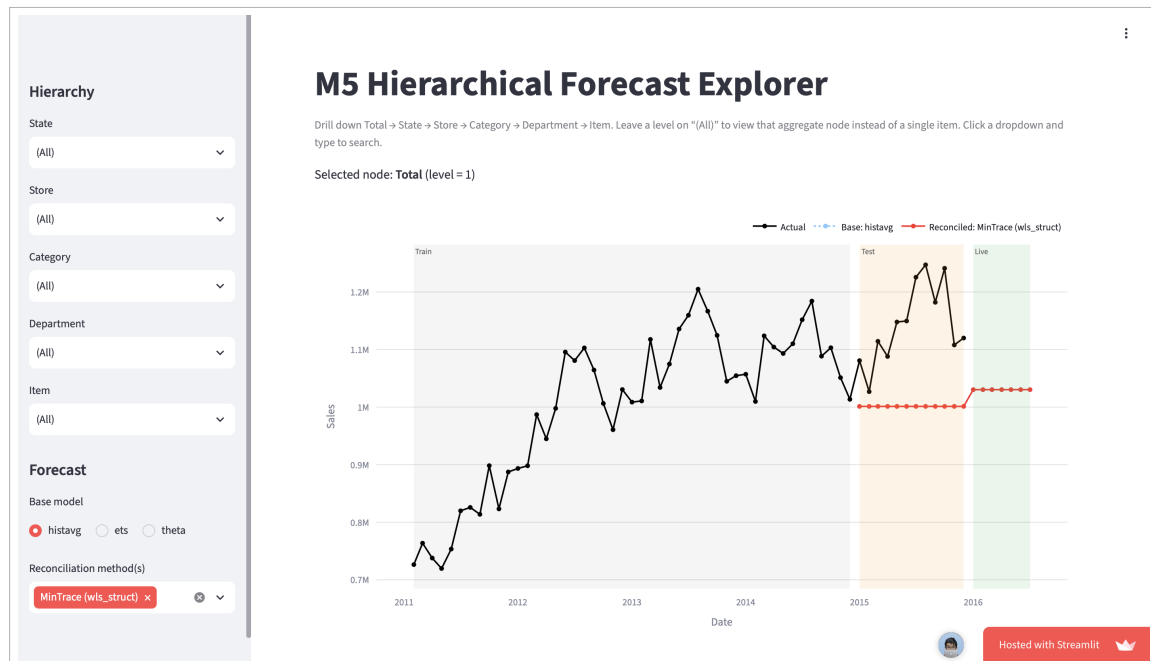


Figure 1.2: The dashboard directly after opening, showing the grand total

1.6 Target Users

- **Examiners and readers of the thesis**, who want to check the reported results for themselves at any level of the hierarchy.
- **Business users** such as category or store managers, who want to see how reliable forecasts are for their part of the business and which reconciliation method suits it.
- **Students and analysts**, who want an intuitive picture of what hierarchical reconciliation does to a forecast.
- **The operator** (the author or a successor), who keeps the online installation running (Chapter 7).

1.7 Structure of this Manual

Table 1.2: Chapters of this manual and who needs them

Chapter	Content	Reader
2 Installation	Open online or install locally	everyone
3 First Steps	A guided first session of 15 minutes	everyone
4 User Interface	Sketch and description of every element	everyone
5 Functions	Each function in detail, with examples	everyone
6 Specifications	Requirements, data, models, limits	everyone
7 Maintenance	Updating and redeploying	operator
8 Troubleshooting	Symptoms, causes and fixes	everyone / operator
9 Help	Frequently Asked Questions, glossary, quick reference	everyone

Conventions. Labels exactly as they appear on the screen are written as **Base model**; values that are selected or typed are written as **CA_1**; terminal commands are shown in grey boxes. Numbers in red circles such as **3** refer to the Graphical User Interface (GUI) sketch in Figure 4.1. Boxes marked *Note*, *Tip* and *Caution* give additional information, shortcuts and warnings.

2 Installation

There are two ways to use the dashboard (Figure 2.1). **Cloud access** (Section 2.1) needs nothing but a web browser and is the right choice for almost every user. **Local installation** (Section 2.2) runs the same dashboard on your own computer; it is needed only to work offline, to check the installation independently of the cloud, or to prepare changes (Chapter 7).

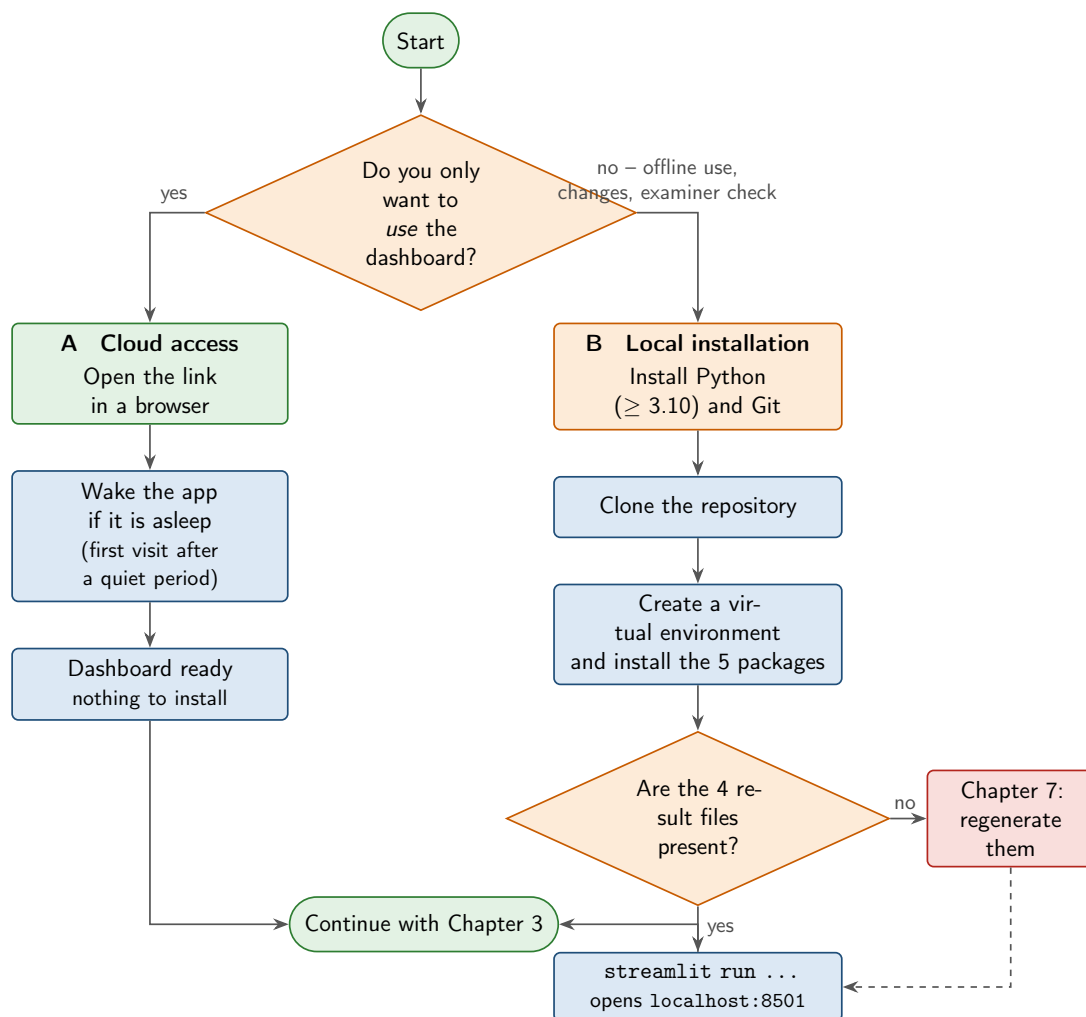


Figure 2.1: Choosing between cloud access and local installation

2 Installation

2.1 Option A: Cloud Access (Recommended)

2.1.1 Requirements

- A current version of Chrome, Firefox, Edge or Safari with JavaScript enabled.
- An internet connection.
- No account, no login, no installation.

2.1.2 Opening the Dashboard

1. Open the browser and enter the address
<https://m5-hierarchical-forecast-explorer.streamlit.app/>
2. Wait until the chart appears (usually 2–5 seconds, Figure 2.2). While the result files are being loaded, Streamlit shows a small running indicator.
3. Bookmark the page for later use.

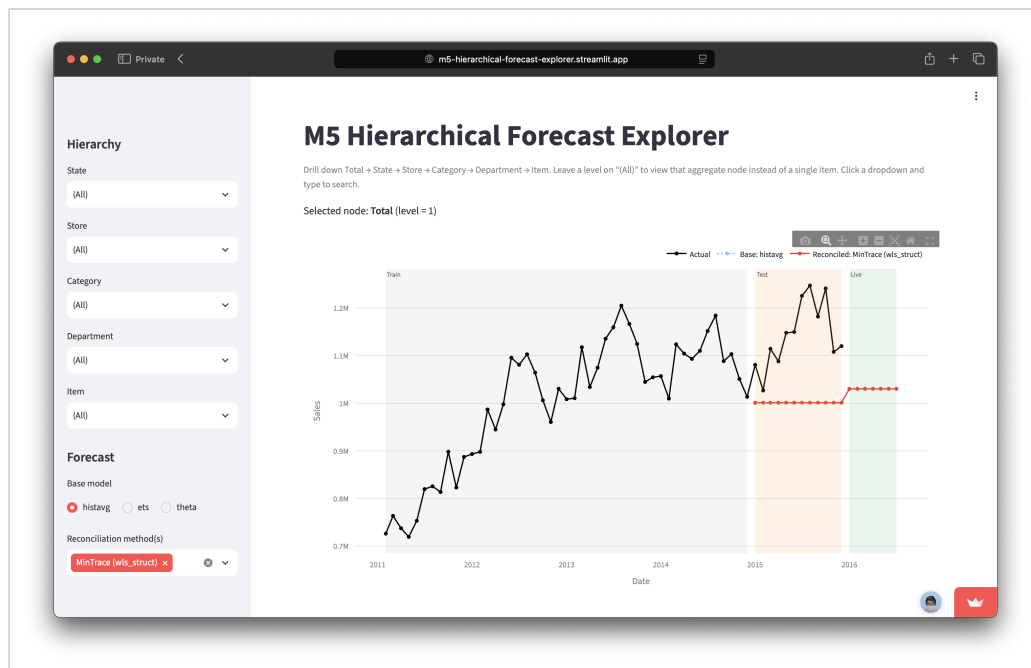


Figure 2.2: The dashboard opened from Streamlit Community Cloud

2.1.3 Waking the App Up

Streamlit Community Cloud puts apps to sleep when nobody has used them for a while [Sno26a]. The first visitor after such a quiet period sees the page in Figure 2.3 instead of the dashboard.

1. Click the button **Yes, get this app back up!**.
2. Wait 30–60 seconds while the app starts. The page reloads by itself.
3. The dashboard appears as usual. It stays awake for all users as long as it is being used.

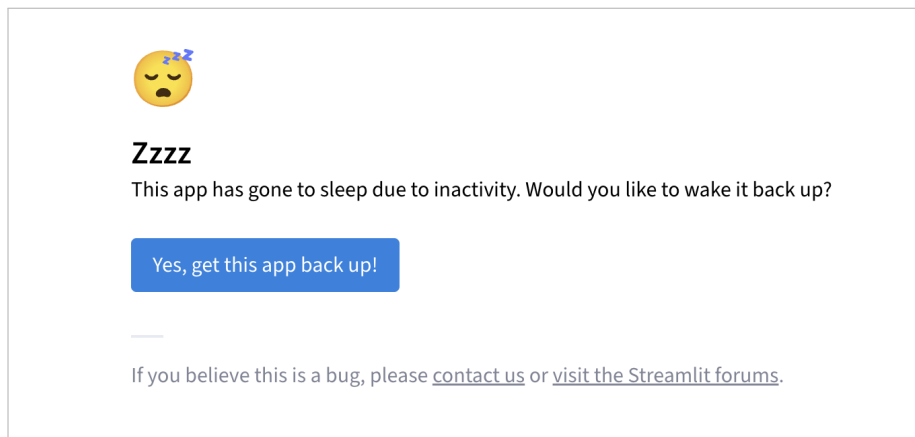


Figure 2.3: The page shown when the app is asleep

Note

Waking the app up does not change anything. The data and forecasts are the same as before; only the program is started again.

2.2 Option B: Local Installation

2.2.1 Requirements

Table 2.1: Requirements for a local installation

Item	Requirement
Operating system	Windows 10/11, macOS 12 or newer, or a current Linux distribution
Python	Version 3.10 or newer (the dashboard uses language features introduced in 3.10)
Git	Any current version, to download the repository
Memory	2 GB of free Random Access Memory (RAM) (the four result files are loaded completely)
Disk space	about 500 MB for the repository and the Python packages
Network	Only for the installation; the dashboard itself runs offline

Check the Python and Git versions in a terminal (Terminal on macOS/Linux, PowerShell on Windows):

2 Installation

```
python3 --version # Windows: python --version
git --version
```

2.2.2 Step 1 – Download the Repository

```
git clone https://github.com/Hemanth-Jp/MBIDA_Master_Thesis.git
cd MBIDA_Master_Thesis
```

All following commands are run from this folder, the *repository root*.

Stay in the repository root

Always start the dashboard from the repository root, not from inside `Code/app/`. Only then does Streamlit pick up the dashboard's settings file `.streamlit/config.toml`, which lives in the root folder – exactly as on Streamlit Community Cloud.

2.2.3 Step 2 – Create a Virtual Environment

A virtual environment keeps the dashboard's packages separate from everything else on the computer.

```
python3 -m venv .venv

# activate it - macOS / Linux:
source .venv/bin/activate
# activate it - Windows (PowerShell):
.venv\Scripts\Activate.ps1
```

After activation, the prompt starts with `(.venv)`. If you prefer conda, the equivalent is `conda create -n m5_dashboard python=3.10` followed by `conda activate m5_dashboard`.

2.2.4 Step 3 – Install the Packages

The dashboard needs only five packages. Their exact versions are listed in `Code/app/requirements.txt` – the same file Streamlit Community Cloud installs from (Section 6.2).

```
pip install -r Code/app/requirements.txt
```

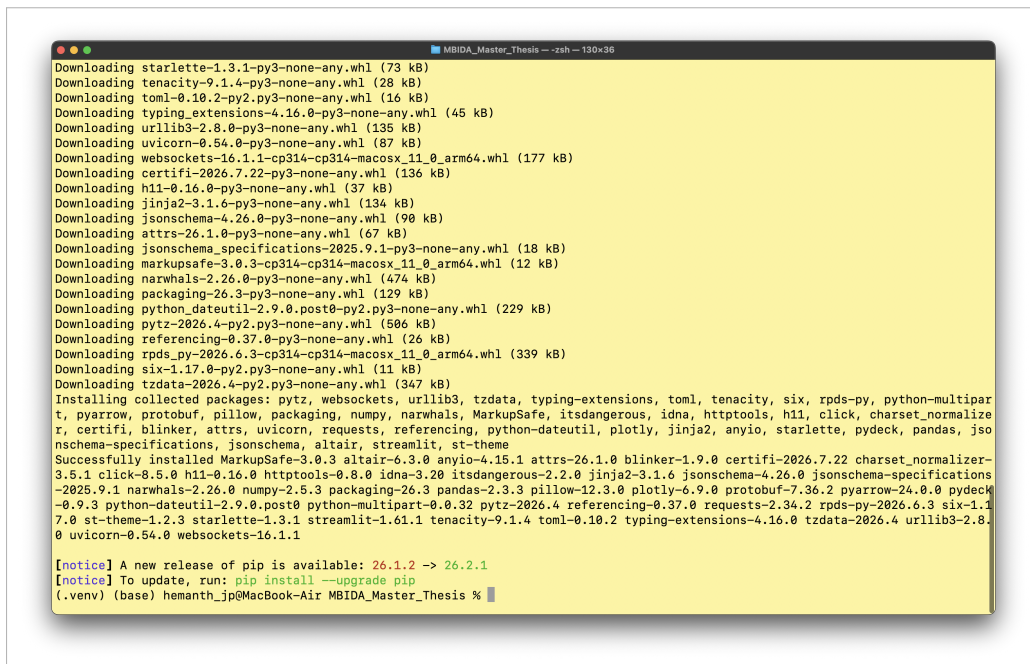


Figure 2.4: Successful installation of the five packages

Package name

The package that lets the chart follow the light/dark theme is called `st-theme`. A different package with the similar name `streamlit-theme` exists and does *not* work. Always install from the requirements file instead of typing package names by hand.

2.2.5 Step 4 – Check the Result Files

The dashboard only displays results; it needs the four result files in `Code/artifacts/dashboard_data/`. They are part of the repository and are downloaded with it:

```
ls Code/artifacts/dashboard_data # Windows: dir Code\artifacts\
    dashboard_data
```

The listing must show `actuals.parquet`, `hierarchy.parquet`, `pred_with_gt.parquet` and `rec_with_gt.parquet`. If one of them is missing, regenerate them once as described in Section 7.7.

2.2.6 Step 5 – Start the Dashboard

```
streamlit run Code/app/streamlit_app.py
```

2 Installation

Streamlit prints the local address and opens it in the default browser (Figures 2.5 and 2.6). On the very first start, Streamlit may ask for an e-mail address in the terminal; simply press **Enter** to skip.

```
MBIDA_Master_Thesis — python3 ~/Documents/GitHub/MBIDA_Master_Thesis/venv/bin/streamlit run Code/app/streamlit_app.py
(.venv) (base) hemanth_jp@MacBook-Air MBIDA_Master_Thesis % ls Code/artifacts/dashboard_data
actuals.parquet      hierarchy.parquet    pred_with_gt.parquet  rec_with_gt.parquet
(.venv) (base) hemanth_jp@MacBook-Air MBIDA_Master_Thesis % streamlit run Code/app/streamlit_app.py
2026-09-26 13:39:00.565 Uvicorn server started on :::8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.20.10.3:8501

Help agents write better Streamlit apps?
Install the official Streamlit skills by running streamlit skills in your terminal.

For better performance, install the Watchdog module:

$ xcode-select --install
$ pip install watchdog
```

Figure 2.5: Terminal output after starting the dashboard locally

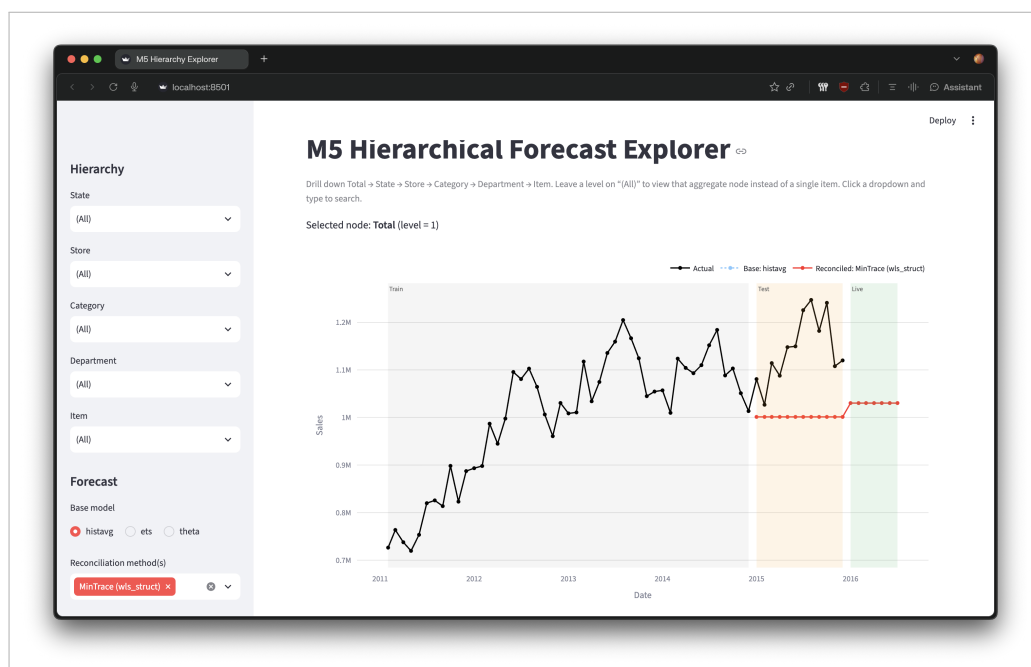


Figure 2.6: The local dashboard at <http://localhost:8501>

To stop the dashboard, press **Ctrl+C** in the terminal. To start it again later, open a terminal in the repository root, activate the environment (Step 2) and repeat Step 5.

2.3 Verifying the Installation

The following check takes one minute and works for both options. If a step fails, go to the section named in the last column.

Table 2.2: Installation check

#	Action	Expected result	If not
1	Open the dashboard	Title M5 Hierarchical Forecast Explorer , no red error box	8.2.1
2	Look at the node line	Selected node: Total (level = 1)	8.4
3	Look at the chart	Black actual line, one dotted line, one solid line; three shaded regions	8.3.1
4	Set State to TX	Chart changes, node line shows Total/TX	8.4
5	Set State back to (All)	Grand total is shown again	–

Tip

The keyboard keys in this manual (**Ctrl+C**, **Enter**) are the Windows/Linux names. On macOS, **Ctrl+C** in the terminal is the same key combination (Control, not Command).

3 First Steps

This chapter is a guided first session of about 15 minutes. It walks through the eight steps in Figure 3.1: first navigating from the grand total down to a single product, then comparing forecasts for it. No previous knowledge is needed; every control used here is explained in detail in Chapters 4 and 5.

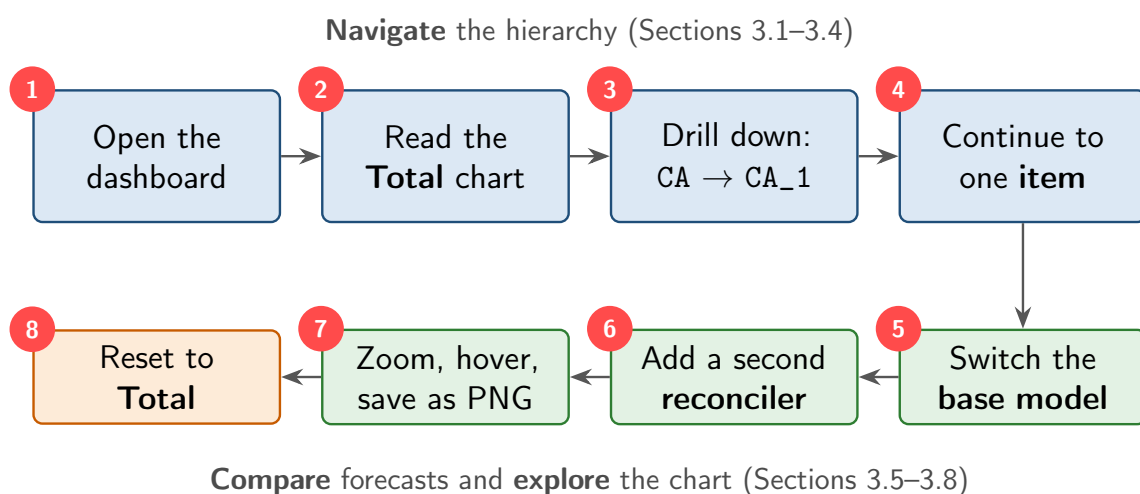


Figure 3.1: The eight steps of the first session

3.1 Step 1 – Open the Dashboard

Open <https://m5-hierarchical-forecast-explorer.streamlit.app/> in the browser, or start your local installation (Chapter 2). If the “gone to sleep” page appears, wake the app up as described in Section 2.1.3.

When the dashboard is ready, all five hierarchy dropdowns show (All), the base model is `histavg` and the reconciliation method is `MinTrace (wls_struct)`. This is the starting position used in the rest of this chapter.

3.2 Step 2 – Read the Total Chart

The first chart (Figure 3.2) shows the grand total: the monthly sales of all 3,024 products in all ten stores added together. Take a moment to read it from left to right:

1. The **grey region (Train)**, February 2011 to December 2014, shows the history the models learned from. The black line with dots is the actual sales.

3 First Steps

2. The **orange region (Test)**, the year 2015, is the most important part. Here the forecasts (coloured lines) can be compared with what was really sold (black line). The closer they are, the better the forecast.
3. The **green region (Live)**, January to July 2016, shows forecasts only. There is no black line here: the live window is kept forecast-only, so the real sales for January to April 2016 are not drawn, and from May 2016 the data set contains no sales at all (Section 6.5).

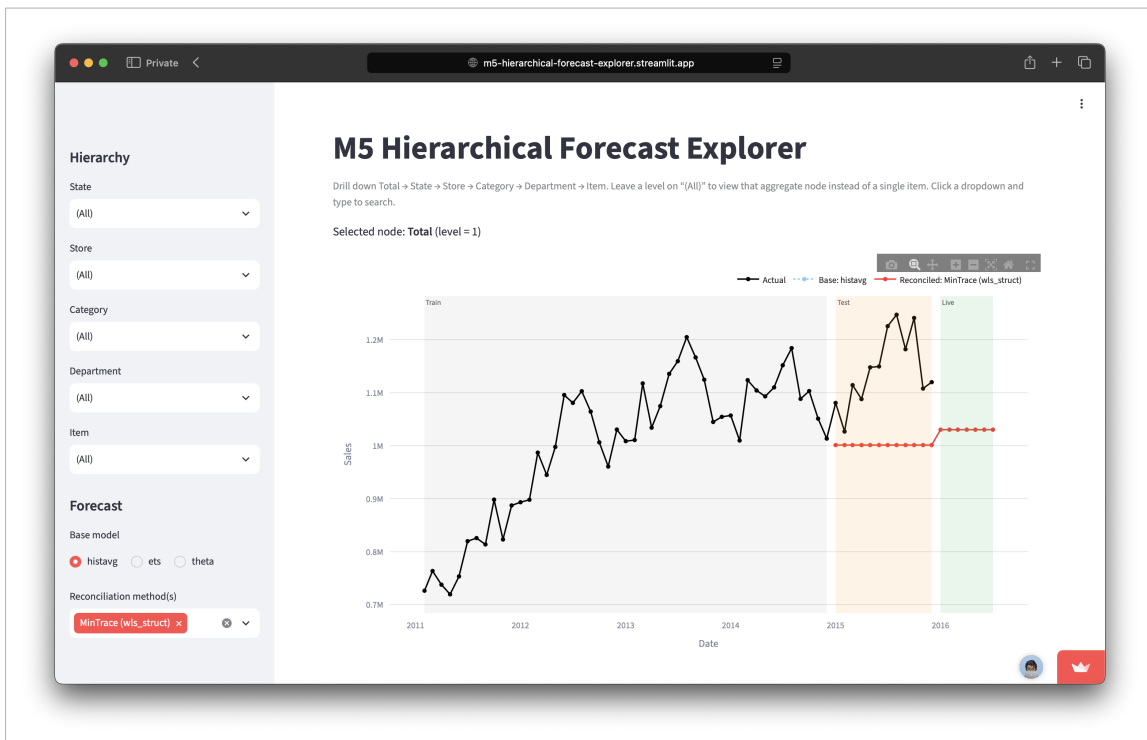


Figure 3.2: The grand total with the default settings

Tip

Move the mouse over any point of a line. A small box shows the date and the exact value of every line at that month.

3.3 Step 3 – Drill Down to a State and a Store

1. In the sidebar, click the **State** dropdown and choose **CA** (California). The chart now shows all Californian stores together, and the line above the chart reads **Selected node: Total/CA (level = 2)**.
2. The **Store** dropdown, which was locked before, now lists the four Californian stores. Choose **CA_1**. The node line changes to **Total/CA/CA_1 (level = 3)** (Figure 3.3).

3.4 Step 4 – Continue to a Single Item

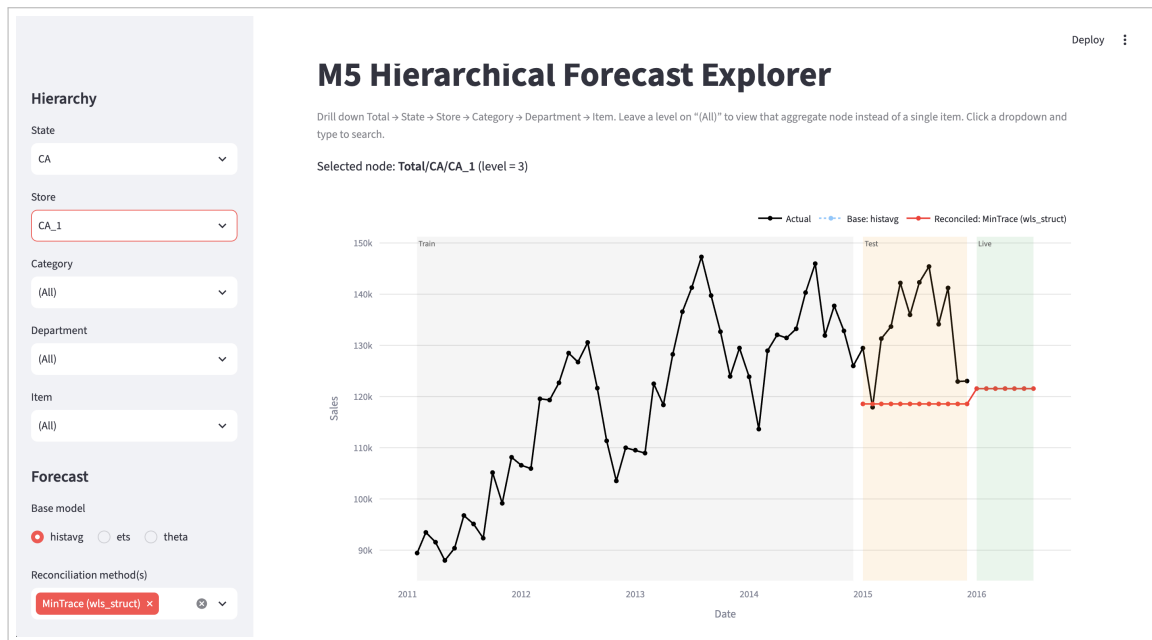


Figure 3.3: Store CA_1 selected; category, department and item are still (All)

Notice that the numbers on the vertical axis are smaller now – one store sells only a part of the total – and that the lines are more irregular. The further down the hierarchy, the more “noisy” sales become, and the harder they are to forecast.

3.4 Step 4 – Continue to a Single Item

1. Set **Category** to FOODS.
2. Set **Department** to FOODS_3.
3. Click the **Item** dropdown and *type* 090 instead of scrolling. The list shrinks to the matching item; choose FOODS_3_090.

The chart now shows one product in one store (level 6, Figure 3.4). This is the finest level of detail in the dashboard.

3 First Steps

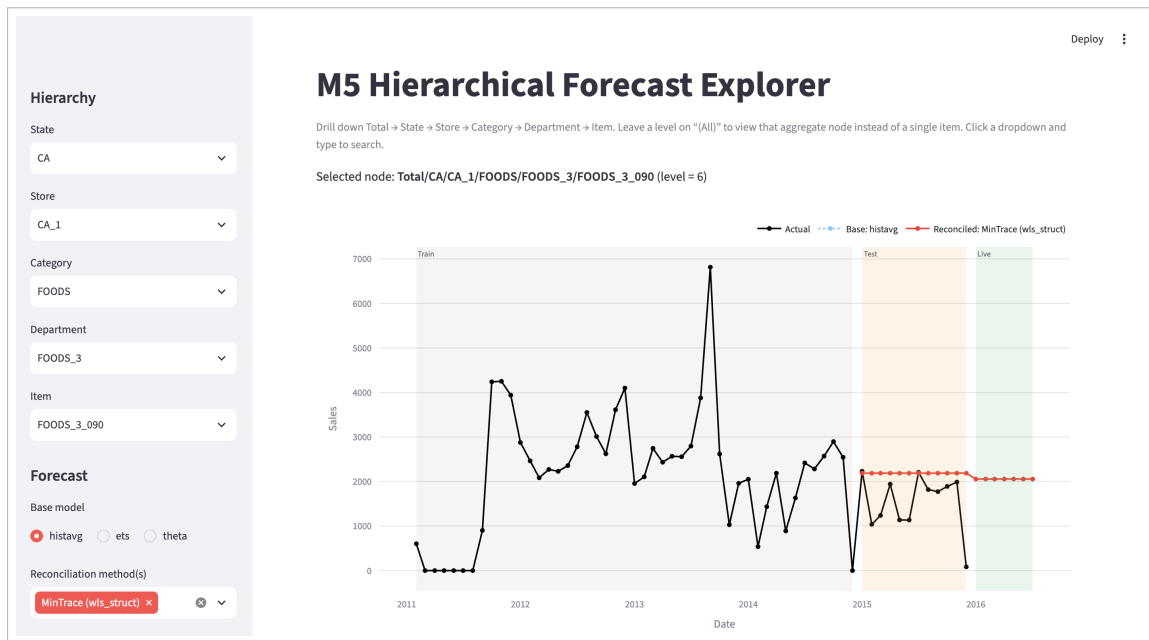


Figure 3.4: A single item, FOODS_3_090 in store CA_1

3.5 Step 5 – Switch the Base Model

Under **Forecast** in the sidebar, the **Base model** buttons select which of the three models provides the dotted line. Click **ets**, then **theta**, and watch how the dotted line changes in the test and live regions. The historical average (**histavg**) is a flat line by nature; **ets** and **theta** follow the yearly pattern.

3.6 Step 6 – Add a Second Reconciliation Method

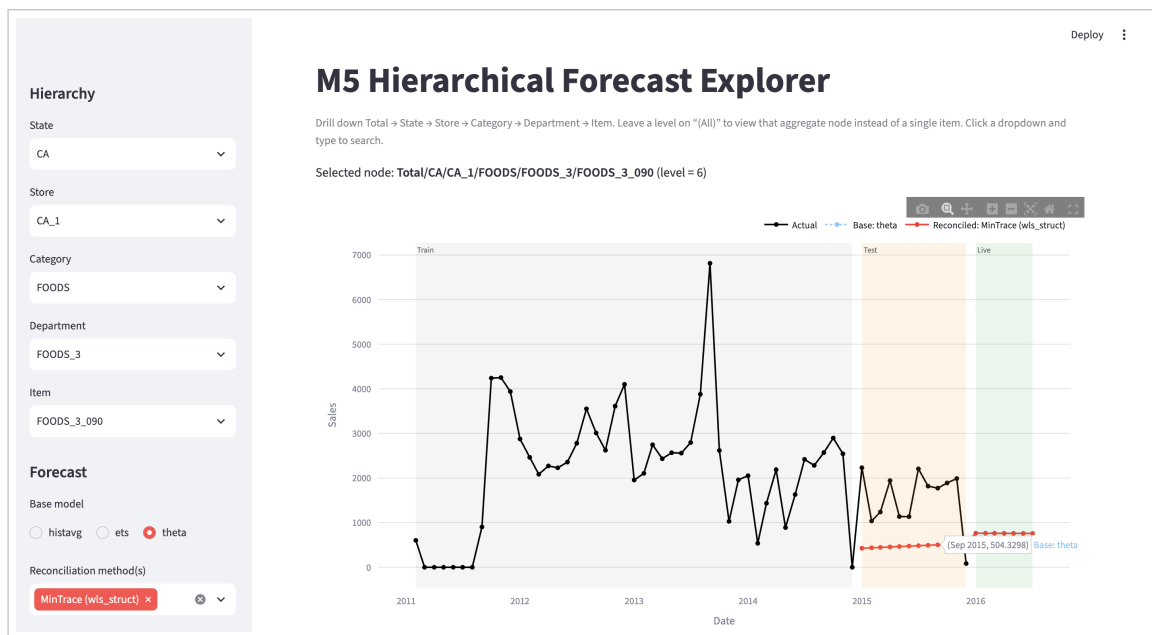


Figure 3.5: The same node with the base model **theta**

Note

The reconciled line changes too, because every reconciled forecast is an adjusted version of the selected base model.

3.6 Step 6 – Add a Second Reconciliation Method

1. Click into the **Reconciliation method(s)** box. A list with the methods that are not selected yet opens.
2. Choose **Bottom-Up**. A second solid line appears, and the legend above the chart names it.
3. Add **Top-Down** as well, so that all three methods are compared in one chart (Figure 3.6).

3 First Steps

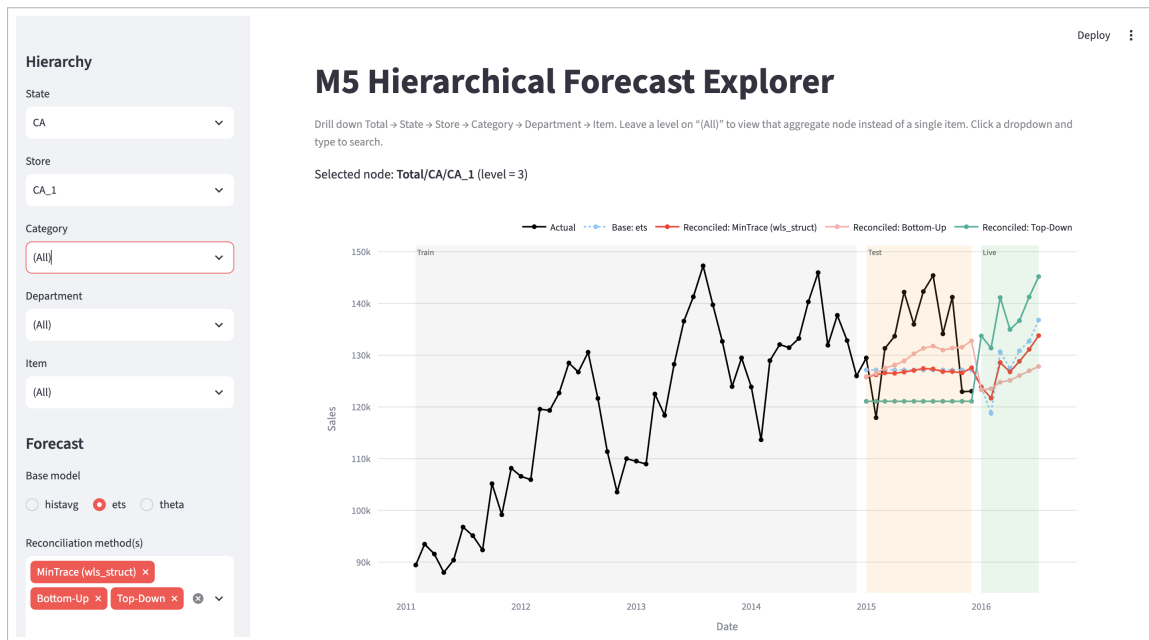


Figure 3.6: All three reconciliation methods compared for one node

To remove a method again, click the small $\times$ on its tag in the box.

3.7 Step 7 – Explore the Chart

- **Zoom:** drag a rectangle over the test region to enlarge it. Double-click the chart to zoom back out.
- **Hide a line:** click an entry in the legend. Click it again to show it.
- **Save the chart:** move the mouse over the chart, then click the camera symbol in the small toolbar at the top right. The chart is saved as a Portable Network Graphics (PNG) file in your download folder.

3.8 Step 8 – Reset to the Total

Set **State** back to (All). All dropdowns below it return to (All) automatically and the grand total is shown again. Reloading the browser page (**F5**) resets every control to its default.

3.9 Summary

After this session you can

- open the dashboard and recognise the three time regions,

- move to any node of the hierarchy with the five dropdowns and the search,
- compare base models and reconciliation methods for that node, and
- zoom, hide lines and save the chart.

This covers the whole everyday use of the dashboard. The following chapters describe each element and function in more depth.

4 User Interface

The dashboard consists of a single page. There are no menus to open, no tabs and no pop-up windows: every control is visible at all times, and every change takes effect immediately. This chapter first presents a sketch of the complete screen and then describes its two parts, the sidebar and the chart.

4.1 Sketch of the User Interface

Figure 4.1 is a schematic sketch of the screen. The numbered elements are listed in Table 4.1; the same numbers are used throughout the manual.

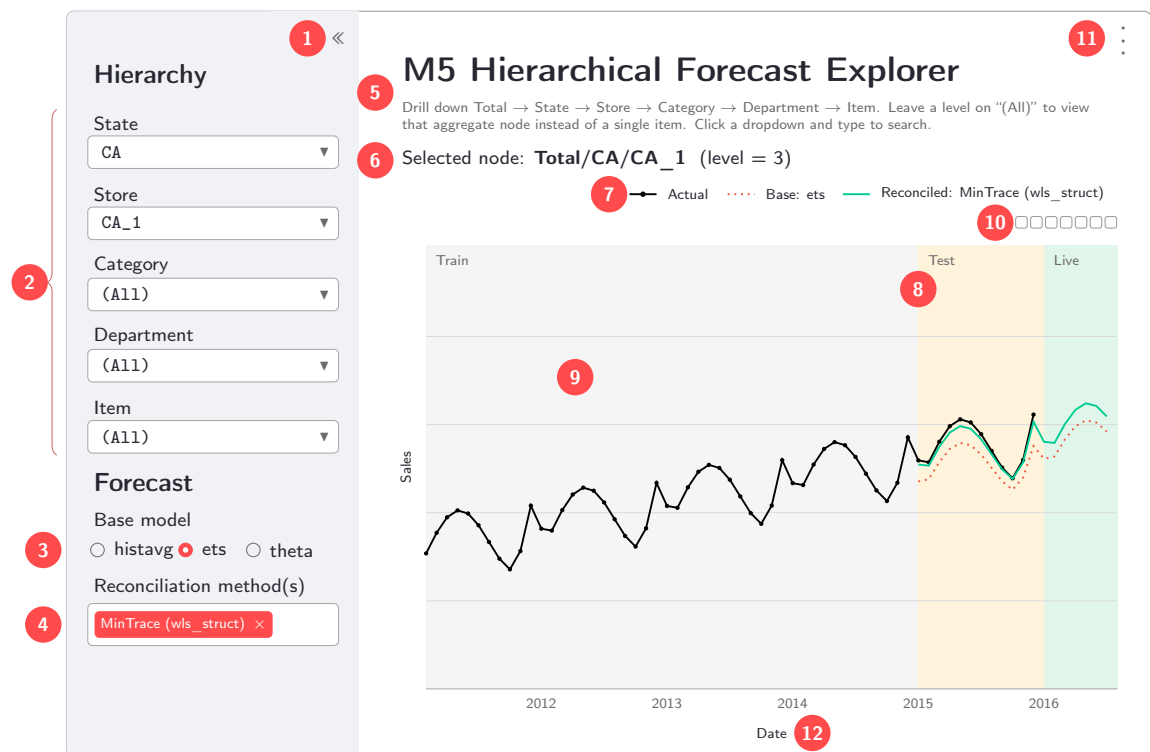


Figure 4.1: Sketch of the user interface with numbered elements

4 User Interface

Table 4.1: Elements of the user interface (numbers as in Figure 4.1)

No.	Element	Purpose
1	Sidebar button	Hides or shows the sidebar, giving the chart the full width
2	Hierarchy dropdowns	Select the node: State , Store , Category , Department , Item
3	Base model	Choose histavg , ets or theta for the dotted line
4	Reconciliation method(s)	Choose one, several or no reconciled lines
5	Title and instructions	Name of the dashboard and a two-line usage hint
6	Node line	The node currently shown and its level (1–6)
7	Legend	Names of all lines; click to hide or show a line
8	Time regions	Shaded Train , Test and Live periods
9	Plot area	Actual sales, base forecast and reconciled forecast(s)
10	Chart toolbar	Save as image, zoom, pan, reset (appears on mouse-over)
11	Page menu	Streamlit's menu: theme, rerun, print
12	Axes	Month (horizontal) and units sold (vertical)

Figure 4.2 shows the same screen as it appears in the browser.

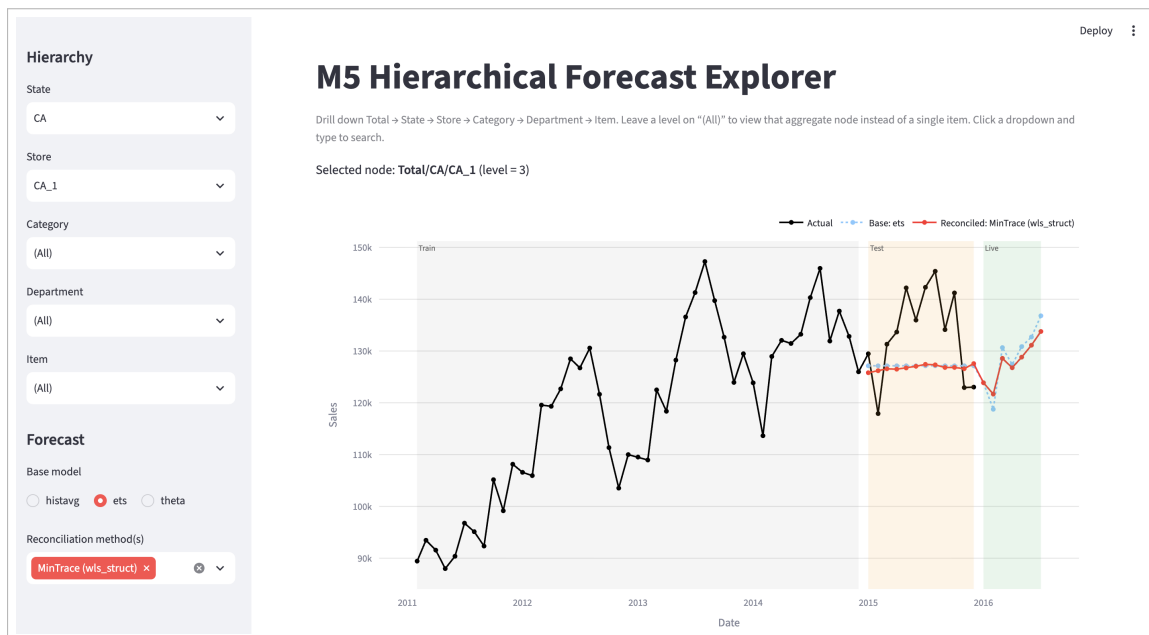


Figure 4.2: The real user interface for store CA_1 with base model ets

4.2 Layout Principles

- **Controls left, result right.** Everything the user can change is in the sidebar; everything the dashboard shows is in the main area.

- **Top to bottom equals coarse to fine.** The dropdowns follow the hierarchy from State down to Item, so the sidebar is read in the same order as the hierarchy.
- **One chart, one node.** The chart always shows exactly one node. Comparing nodes is done by switching between them.
- **Always consistent.** A dropdown only offers values that exist below the choices made above it, so an impossible combination cannot be selected.

4.3 The Sidebar

4.3.1 Hierarchy Section

The upper part of the sidebar 2 holds one dropdown per hierarchy level (Figure 4.3, left).

Hierarchy

State
CA ▼

Store
CA_1 ▼

Category
(All) ▼

Department
(All) ▼

Item
(All) ▼

Forecast

Base model

☐ histavg
 ☒ ets
 ☐ theta

Reconciliation method(s)

MinTrace (wls_struct) ×

Bottom-Up ×

Top-Down ×

× ▼

Figure 4.3: Sidebar: hierarchy section (left) and forecast section (right)

The five dropdowns are linked:

Table 4.2: Hierarchy dropdowns

Dropdown	Values	Available when
State	(A11), CA, TX, WI	always
Store	(A11) + stores of the chosen state, e.g. CA_1 to CA_4	a state is chosen
Category	(A11), FOODS, HOBBIES, HOUSEHOLD	a store is chosen
Department	(A11) + departments of the category, e.g. FOODS_1 to FOODS_3	a category is chosen
Item	(A11) + items of the department in that store	a department is chosen

As long as a dropdown is on (A11), all dropdowns below it offer only (A11). Each dropdown can be searched by typing (Section 5.3).

4.3.2 Forecast Section

The lower part of the sidebar holds the two forecast controls 3 and 4 (Figure 4.3, right).

Base model 3

Three round buttons in one row. Exactly one is active; the default is `histavg`.

Reconciliation method(s) 4

A multi-selection box. Selected methods appear as red tags with a $\times$ to remove them. Clicking into the box lists the remaining methods. The default is `MinTrace` (`wls_struct`). The box may also be left empty.

4.3.3 Collapsing the Sidebar

On small screens the sidebar takes a lot of space. The double arrow 1 at its top right hides it; the chart then uses the full window width. A small arrow at the top left of the page brings the sidebar back. On phones the sidebar is hidden by default.

4.4 The Main Area

4.4.1 Title, Instructions and Node Line

Below the title 5, a short grey text summarises how to use the dropdowns. The line directly above the chart, the *node line* 6, always states which node is shown:

```
Selected node: Total/CA/CA_1 (level = 3)
```

The node name lists the path through the hierarchy, separated by slashes; the level counts from 1 (grand total) to 6 (single item). If a selected reconciliation method cannot be found in the result files, a warning is appended to this line (Section 8.2.3).

4.4.2 The Forecast Chart

Figure 4.4 explains how to read the plot area **9** with its time regions **8** and axes **12**. The data in the sketch is illustrative; the structure is exactly that of the real chart.

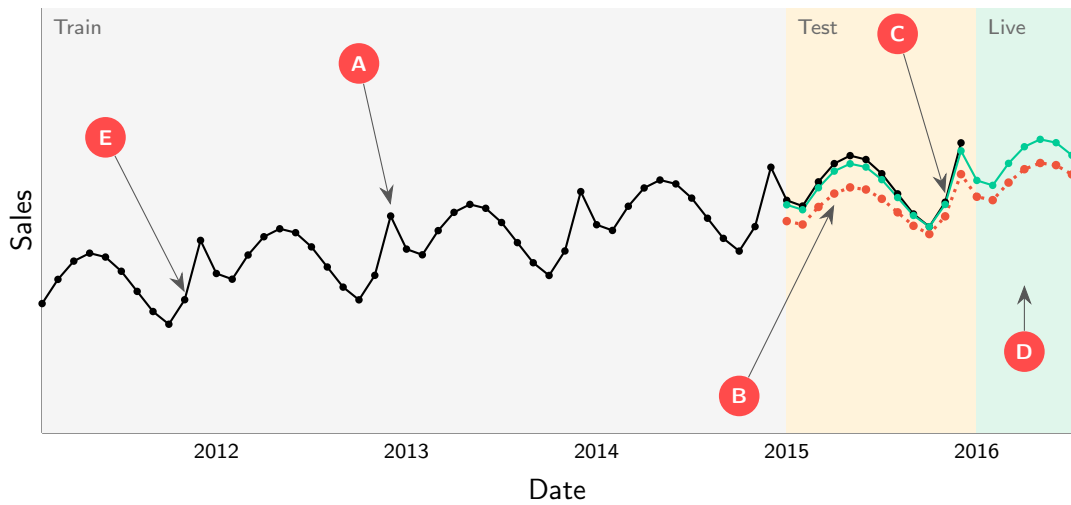


Figure 4.4: Anatomy of the forecast chart (illustrative data)

Table 4.3: Lines and regions of the chart (letters as in Figure 4.4)

Element	Meaning
A Actual (black, solid, dots)	Units really sold per month; drawn over train and test only
B Base (dotted)	Forecast of the selected base model; test and live only
C Reconciled (solid, coloured)	One line per selected reconciliation method; test and live only
D Live region	Forecasts without an actual line – no real sales to compare with
E Yearly peak	Seasonal December peak; a good forecast repeats it

Line colours are assigned automatically in a fixed order; the legend **7** is always the reference for which colour belongs to which method. The legend reads, for example, **Base: ets** and **Reconciled: MinTrace (wls_struct)**.

4.4.3 Chart Toolbar

When the mouse is over the chart, a small toolbar ¹⁰ appears at its top right (Figure 4.5). It is provided by the charting library Plotly [Plo26].

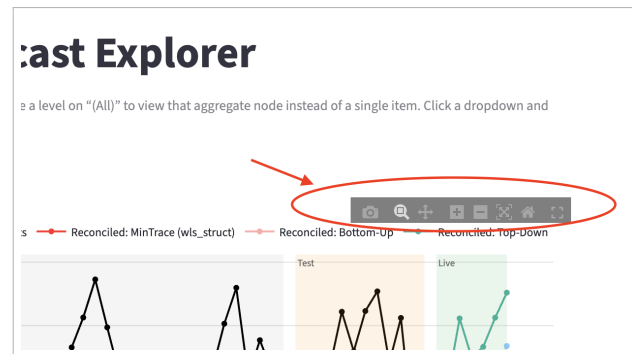


Figure 4.5: The chart toolbar

Table 4.4: Chart toolbar buttons

Button	Function
Camera	Download the current view of the chart as PNG
Magnifier	Zoom mode: drag a rectangle to enlarge it (default mode)
Cross arrows	Pan mode: drag to move the visible range
Plus / minus	Zoom in / out around the centre
Autoscale	Fit all lines into the view
House	Reset the axes to the original view
Expand (Streamlit)	Separate button next to the toolbar: show the chart in full screen

4.5 Page Menu and Theme

The three dots ¹¹ at the top right of the page open Streamlit's own menu [Sno26b]. For users, two entries matter:

- **Settings** – choose **Light**, **Dark** or **Use system setting**. The chart adjusts its colours automatically (Figure 4.6).
- **Print** – prints the page including the current chart.

On a local installation, a **Deploy** button also appears next to the menu; it is not needed and can be ignored.

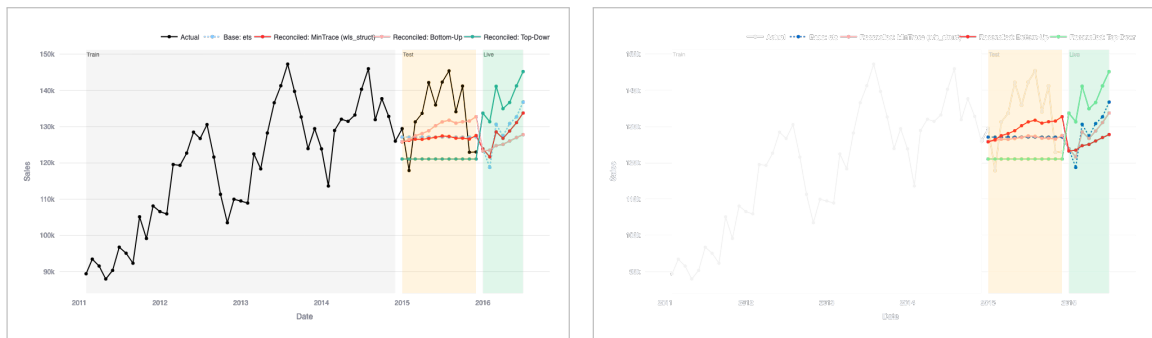


Figure 4.6: The same chart in light mode (left) and dark mode (right)

4.6 Messages

The dashboard uses Streamlit's standard message boxes. In normal use none of them appears.

Table 4.5: Messages the dashboard can show

Type	Beginning of the text	See
Red error box	Couldn't find one of the required parquet files	Section 8.2.1
Yellow warning	No data found for '...'	Section 8.2.2
Node line suffix	No column found for: ...	Section 8.2.3
Running indicator	(animation, top right)	normal while load- ing

5 Functions

This chapter describes each function of the dashboard: what it does, how it is used and what to look out for. The functions fall into three groups – *choosing what to look at* (Sections 5.1–5.3), *choosing which forecasts to compare* (Sections 5.4–5.5) and *exploring the chart* (Sections 5.6–5.7). Section 5.8 closes with typical analysis tasks that combine them.

5.1 The Sales Hierarchy

All navigation in the dashboard follows the six-level hierarchy in Figure 5.1. Every node is the sum of the nodes directly below it: the three states add up to the total, the four Californian stores add up to CA, and so on down to the 30,240 single products-in-a-store at the bottom.

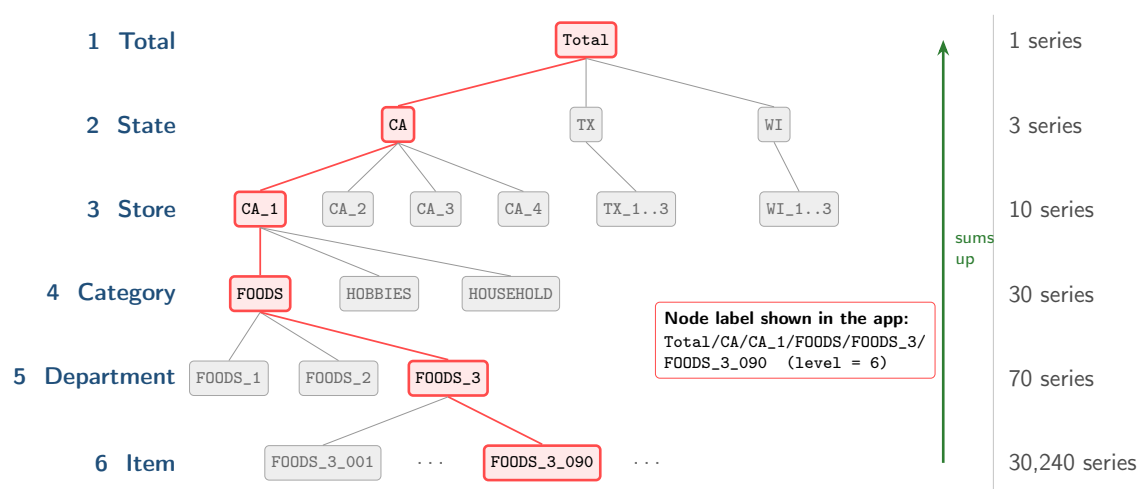


Figure 5.1: The six-level sales hierarchy with the example path used in this manual

Table 5.1: Levels of the hierarchy

Level	Name	Nodes	Example
1	Total	1	all sales
2	State	3	CA (California), TX (Texas), WI (Wisconsin)
3	Store	10	CA_1 ... CA_4, TX_1 ... TX_3, WI_1 ... WI_3
4	Category	30	FOODS in store CA_1
5	Department	70	FOODS_3 in store CA_1
6	Item	30,240	product FOODS_3_090 in store CA_1
Sum		30,354	

Categories are always inside a store

The hierarchy is strictly *store first, then category*. The dashboard therefore shows “FOODS in store CA_1”, but not “FOODS in all stores together”. Such a cross-store view is not one of the 30,354 nodes (Section 6.9).

5.2 Navigating with the Cascading Dropdowns

What it does. The five dropdowns  select the node. The dashboard reads them from top to bottom and stops at the first one that is left on (A11); the node is everything chosen above that point (Figure 5.2).

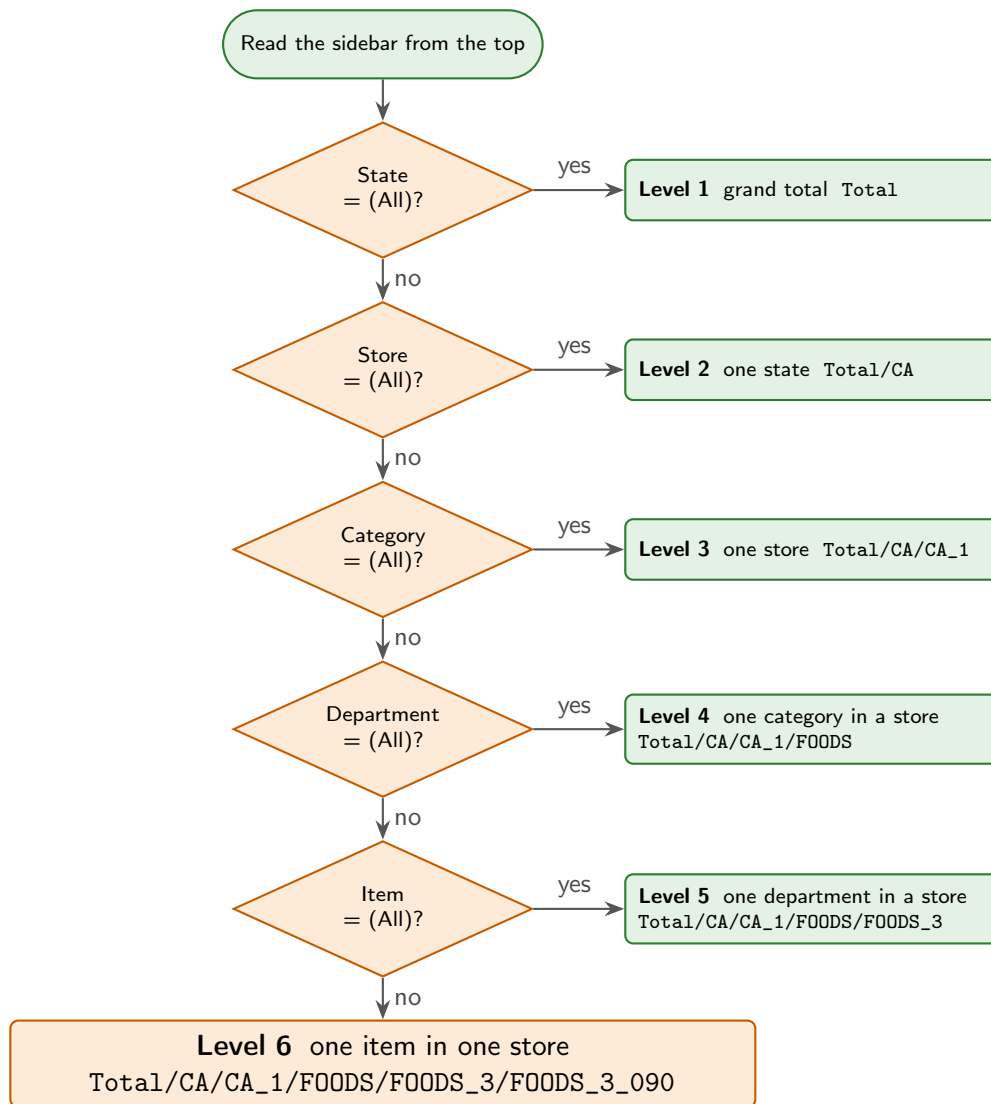
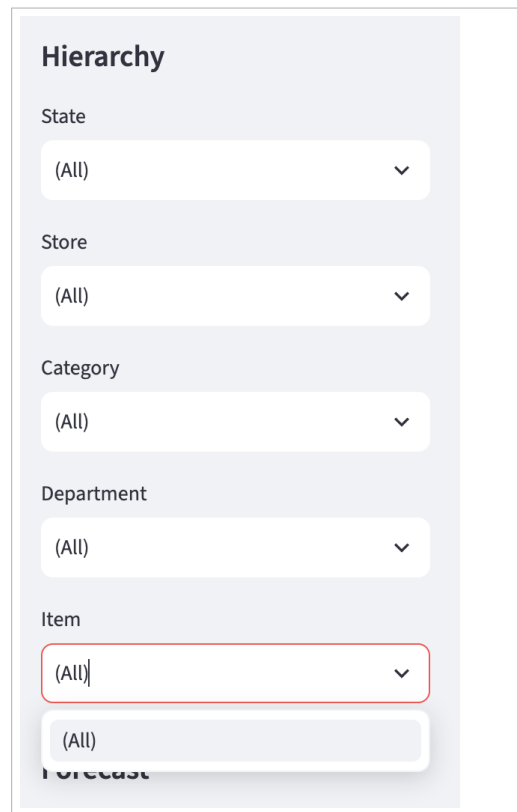


Figure 5.2: How the five dropdowns determine the node that is shown

How to use it.

- **Going down:** choose values from top to bottom. Each choice unlocks the next dropdown and restricts it to what exists below.
- **Going up one level:** set the lowest chosen dropdown back to (All).
- **Going back to the total:** set **State** to (All); everything below follows.
- **Going sideways:** change a dropdown to a sibling, e.g. **Store** from CA_1 to CA_2. The dropdowns below it are usually reset to (All), because their lists of values change with the new store.

Locked dropdowns. A dropdown below an (A11) offers only (A11) (Figure 5.3). This is intended: without a store, for example, “department FOODS_3” would be ambiguous.



The image shows a form titled "Hierarchy" with five dropdown menus. The labels for the dropdowns are "State", "Store", "Category", "Department", and "Item". Each dropdown menu currently displays the text "(All)" and has a downward-pointing arrow on the right. The "Item" dropdown is highlighted with a red border. Below the "Item" dropdown, another "(All)" option is visible, suggesting a list of options that is currently filtered to only show "(All)".

Figure 5.3: With **State** on (A11), all lower dropdowns are locked to (A11)

5.3 Searching in a Dropdown

Every dropdown can be searched. Click it and start typing; the list is filtered to entries containing the typed text anywhere in the name (Figure 5.4). Press **Enter** to take the first match or click the wanted entry. This is the fastest way to reach an item: a department contains up to several hundred items per store, which is too many to scroll through.

The screenshot shows a 'Hierarchy' panel with the following settings:

- State: CA
- Store: CA_1
- Category: FOODS
- Department: FOODS_2
- Item: 090 (highlighted with a red border, with a suggestion of FOODS_2_090 shown below it)

Figure 5.4: Typing 090 in the **Item** dropdown

Tip

Item names follow the pattern `CATEGORY_DEPARTMENT_NUMBER`, e.g. `HOUSEHOLD_1_118`. Typing just the number is usually enough, because the department is already fixed by the dropdown above.

5.4 Choosing the Base Model

What it does. The **Base model** buttons **3** select which of three forecasting models draws the dotted line – and which model the reconciled lines are based on. All three were calculated in advance for every node, so switching is instant.

Table 5.2: The three base models

Button	Idea in one sentence	Typical look in the chart
histavg	The forecast is the average of all past months.	A flat line; serves as the simple benchmark every other model must beat.
ets	Exponential smoothing: recent months count more, and trend and yearly season are modelled [HA21].	Follows level, trend and the December peak.
theta	Combines a long-term trend line with a short-term smoothed view [HA21].	Similar to ets , often slightly smoother.

What to look out for. For nodes with too little history to fit **ets** or **theta** – mostly single items that rarely sell – the forecast falls back to the historical average. Such a node shows the same flat line for all three buttons. This is expected and is not an error.

5.5 Choosing Reconciliation Methods

What it does. Base forecasts are made for each node separately, so the forecasts of the parts do not add up to the forecast of the whole. Reconciliation corrects this. The **Reconciliation method(s)** box  adds one solid line per selected method.

Table 5.3: The three reconciliation methods

Method	Idea in plain words
Bottom-Up	Trust the item forecasts and add them up to every higher level.
Top-Down	Trust the total forecast and split it downwards according to each node's historical share of sales.
MinTrace (wls_struct)	A compromise: adjust the forecasts on all levels at once, as little as possible, so that they add up; nodes that consist of many items get more weight [WAH19].

How to use it. Any combination is possible: none (only actual and base line), one, two or all three methods. Comparing all three at once is the main use. Because each reconciled line is the *base forecast after adjustment*, the gap between the dotted line and a solid line shows how much the method changed the forecast for this node.

Reading the effect by level.

- At the **Total** (level 1), **Top-Down** is identical to the base forecast, because it starts from the total.
- At the **Item** level (level 6), **Bottom-Up** is identical to the base forecast, because it starts from the items.

- MinTrace changes every level a little.

5.6 Exploring the Chart

The chart is interactive [Plo26]. Table 5.4 lists all interactions.

Table 5.4: Chart interactions

Action	Result
Mouse over a line	Box with month and value of that point
Drag a rectangle	Zoom into this area
Drag along an axis	Zoom or move only this axis
Double-click	Back to the full view
Click a legend entry	Hide or show this line
Double-click a legend entry	Show only this line; double-click again shows all
Camera button 	Save the current view as PNG

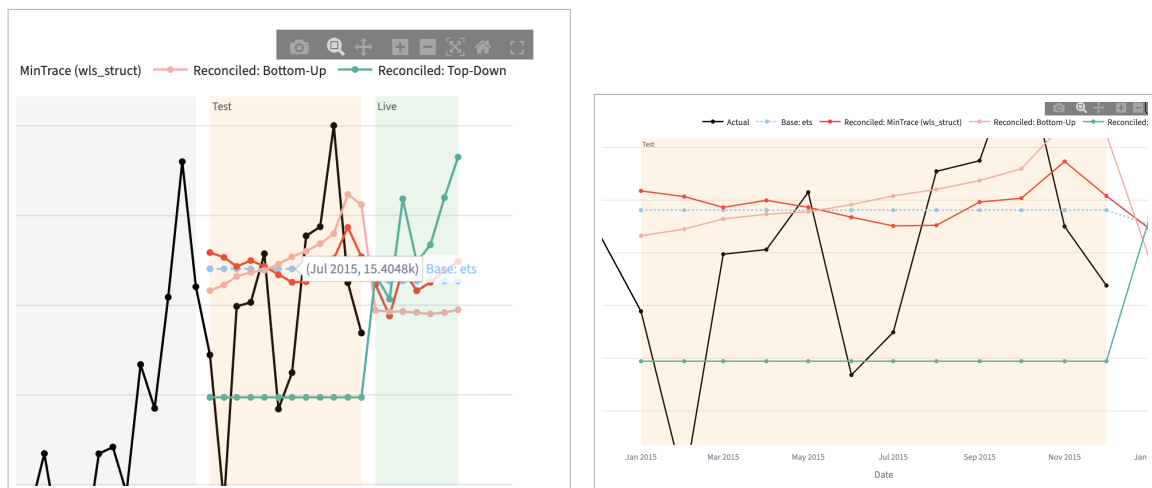


Figure 5.5: Hover information (left) and the test period after zooming in (right)

Note

Zoom and hidden lines are reset whenever another node, model or method is selected, because the chart is drawn anew. Save an image first if the view is needed later.

5.7 Light and Dark Mode

The dashboard follows the theme chosen in the page menu  (Section 4.5). The chart detects the active theme and switches its colours for lines, grid, text and shading, so it stays readable in both modes. With `Use system setting`, the theme changes

together with the operating system's light/dark setting. No separate switch inside the dashboard is needed.

5.8 Typical Analysis Tasks

Table 5.5: Typical tasks and how to do them

Task	Procedure
How good is the forecast for my store?	Select the store, keep MinTrace , try all three base models; compare with the black line in the Test region.
Which reconciler suits a department?	Select the department, one base model, all three methods; hover along the test year.
How much does reconciliation change a forecast?	Compare the dotted base line with the solid lines; the larger the gap, the stronger the adjustment.
Why are item forecasts worse than totals?	Compare the total (level 1) with any item (level 6): item sales are small and irregular, totals are smooth.
Put a chart into a report	Set up the view, zoom if needed, click the camera button; switch to light mode first for printing.

The dashboard shows forecasts visually. The measured accuracy of all models and methods over the test year, per level, is summarised in Section 6.7.

6 Specifications

This chapter collects the technical facts about the dashboard in one place: what it runs on, which data it shows, which time windows, models and methods it covers, how accurate the forecasts were, and where its limits are.

6.1 Overview

Table 6.1: Key facts

Name	M5 Hierarchical Forecast Explorer
Type	Read-only web dashboard (single page)
Online address	https://m5-hierarchical-forecast-explorer.streamlit.app/
Hosting	Streamlit Community Cloud (free tier)
Data source	M5 competition data, Walmart unit sales 2011–2016 [Kag20]
Time resolution	Monthly
Hierarchy	6 levels, 30,354 nodes (Table 5.1)
Base models	3 – <code>histavg</code> , <code>ets</code> , <code>theta</code>
Reconciliation methods	3 – Bottom-Up, Top-Down, MinTrace (<code>wls_struct</code>)
Forecast periods	Test: 12 months (2015); Live: 7 months (Jan–Jul 2016)
User accounts	none; the dashboard asks for no personal data

6.2 Software and Runtime

Figure 6.1 shows the three layers involved at run time: the browser, the server that runs the dashboard program, and the result files it reads.

6 Specifications

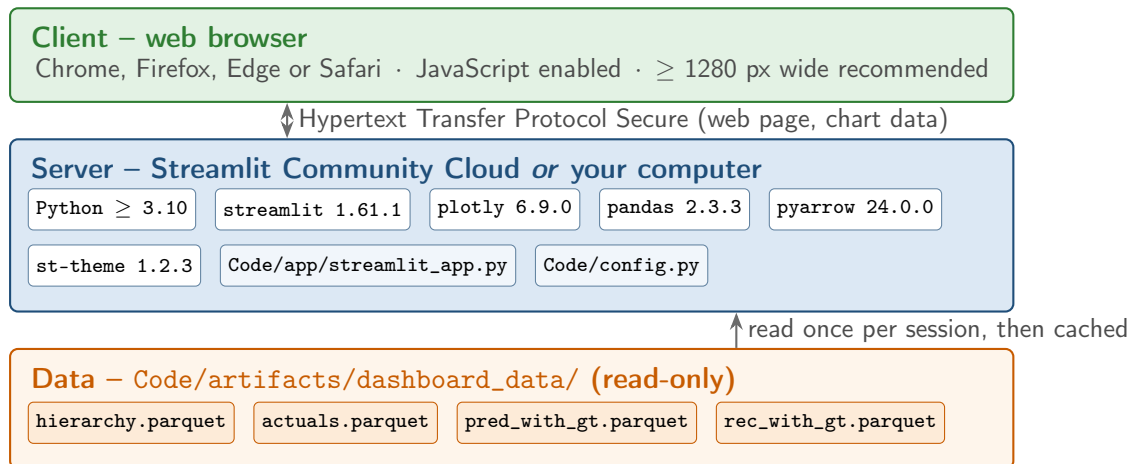


Figure 6.1: Runtime layers of the dashboard

Table 6.2: Software components (pinned versions from Code/app/requirements.txt)

Component	Version	Role
Python	≥ 3.10	Programming language the dashboard is written in
streamlit	1.61.1	Web framework: page, sidebar, dropdowns, hosting [Sno26b]
plotly	6.9.0	Interactive chart [Plo26]
pandas	2.3.3	Filtering the data for the selected node
pyarrow	24.0.0	Reading the Parquet result files
st-theme	1.2.3	Detecting the active light/dark theme

The dashboard itself consists of two program files: Code/app/streamlit_app.py (the page) and Code/config.py (shared settings such as names of models, methods and time windows). None of the forecasting libraries is needed to run it.

6.3 Client Requirements

- Browser: current Chrome, Firefox, Edge or Safari; JavaScript and cookies for the site enabled.
- Screen: works from phone size upwards; a width of at least 1280 pixels is recommended to see sidebar and chart side by side.
- Input: mouse or touchpad for hover and zoom; on touch screens, tap shows values and pinch zooms.

6.4 Data

6.4.1 Source

The data comes from the M5 forecasting competition [MSA22; Kag20]: daily unit sales of 3,049 Walmart products in ten stores in California, Texas and Wisconsin, from 29 January 2011 to 24 April 2016. For the dashboard, the thesis pipeline prepared it once as follows:

- daily sales were added up to **monthly** sales;
- the incomplete first month (January 2011) was removed, so the history starts in **February 2011**;
- 25 products that were not sold at all before 2015 were removed, leaving **3,024 products** and **30,240** product-store combinations;
- months without sales were filled with zero.

Sales are counted in **units** (pieces sold), not in currency.

6.4.2 Result Files

The dashboard reads four result files from `Code/artifacts/dashboard_data/`. They are in the Parquet format, a compact table format, and are stored in the repository.

Table 6.3: Result files read by the dashboard

File	Content (user view)
<code>hierarchy.parquet</code>	The list of all state/store/category/department/item combinations – fills the dropdowns
<code>actuals.parquet</code>	Monthly actual sales for every node, with its time window (train/test/live)
<code>pred_with_gt.parquet</code>	The three base forecasts for every node over the test and live periods
<code>rec_with_gt.parquet</code>	The nine reconciled forecasts (3 models $\times$ 3 methods) for every node and the same periods

The data is static

The result files do not change while the dashboard is running, and the dashboard never writes to them. Every user sees the same numbers. New numbers would require running the thesis pipeline again (Section 7.7).

6.5 Time Windows

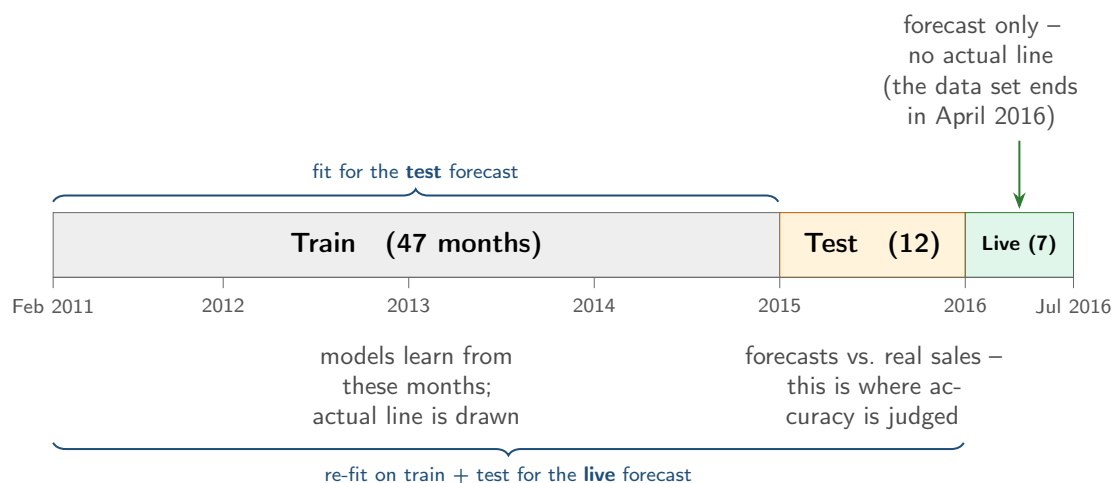


Figure 6.2: The three time windows on a monthly scale

Table 6.4: Time windows

Window	Months	Count	Use in the chart
Train	Feb 2011 – Dec 2014	47	Actual line only; models learned from these months
Test	Jan 2015 – Dec 2015	12	Actual line and all forecasts – the comparison period
Live	Jan 2016 – Jul 2016	7	Forecasts only

The test forecasts were made using the train months only, as if 2015/16 were still unknown. The live forecasts were made using train and test months together. The live window reaches beyond the end of the data set (April 2016).

6.6 Models and Methods

Table 6.5: Forecasting settings

Setting	Value
Base models	HistoricAverage (histavg), AutoETS (ets), Theta (theta)
Season length	12 months (yearly pattern)
Fallback	HistoricAverage, for series where AutoETS or Theta cannot be fitted
Negative forecasts	set to zero before reconciliation (unit sales cannot be negative)
Reconciliation	Bottom-Up; Top-Down with average historical proportions; MinTrace with Weighted Least Squares structural weights and non-negativity [WAH19; OGL+22]
Forecasts per node	3 base + 9 reconciled = 12 lines available, of which up to 4 are shown at once

6.7 Forecast Accuracy (Reference)

To help interpret what the chart shows, Table 6.6 gives the average error of each combination over the test year 2015, measured with the Symmetric Mean Absolute Percentage Error (sMAPE). Lower is better; 0% would be a perfect forecast. For each level, the best value is printed in bold.

Table 6.6: Average sMAPE in % over the test year, by level (**ets** and **theta**)

Level	ets				theta			
	Base	BU	MinT	TD	Base	BU	MinT	TD
1 Total	5.53	3.94	5.19	5.53	4.24	3.78	4.36	4.24
2 State	4.88	3.97	5.11	5.88	4.32	3.76	4.36	4.77
3 Store	5.95	4.77	5.58	7.60	5.35	4.86	5.25	6.97
4 Category	7.31	6.22	7.19	10.63	6.73	6.18	6.52	9.75
5 Department	7.88	7.91	8.00	12.38	8.26	7.60	7.98	11.49
6 Item	37.72	37.72	40.62	40.74	36.32	36.32	39.24	40.54

For **histavg**, the error is 6.58% at the total, rising to 40.91% at item level, and Bottom-Up and Top-Down leave it unchanged. MinTrace leaves it practically unchanged on levels 1 to 5 as well; only at item level does it raise the error, from 40.91% to 41.76%. In short: the higher the level, the more accurate the forecast; for this data, Bottom-Up gave the lowest error on most levels. The thesis discusses these results in detail.

6.8 Performance

Table 6.7: Typical response times

Situation	Typical time
Opening the running online app	a few seconds (the four files are loaded once)
Waking a sleeping online app	about half a minute to a minute
Changing any control	well under a second (data is kept in memory)
Local start (<code>streamlit run</code>)	a few seconds until the browser opens

6.9 Limitations

- **Fixed results.** The dashboard shows pre-computed forecasts; it cannot forecast new periods or new data.
- **Monthly only.** Daily or weekly patterns are not visible.
- **Strict hierarchy.** Categories and departments exist only inside a store. Cross-store views such as “all FOODS in Texas” are not available.
- **No live ground truth.** The live window has no actual line, so forecasts there cannot be judged in the dashboard.
- **No prediction intervals.** Forecasts are single values without an uncertainty band.
- **No table export.** Only the chart image can be downloaded.
- **Free hosting.** The online app may go to sleep (Section 2.1.3) and has limited resources shared by all visitors.

7 Maintenance

This chapter is written for the *operator*: the person who owns the GitHub repository and the Streamlit Community Cloud app. Users of the dashboard do not need it.

Because the data is static, maintenance is small. It consists of keeping the online app available, keeping the software working, and redeploying after a change. The forecasting pipeline is *not* part of regular maintenance; it is only run again if the result files are lost (Section 7.7).

7.1 Maintenance Cycle

Every change to the dashboard – a new package version, a corrected text, a changed setting – follows the same cycle (Figure 7.1). The online app is rebuilt automatically whenever the repository’s main branch changes; there is no separate upload step.

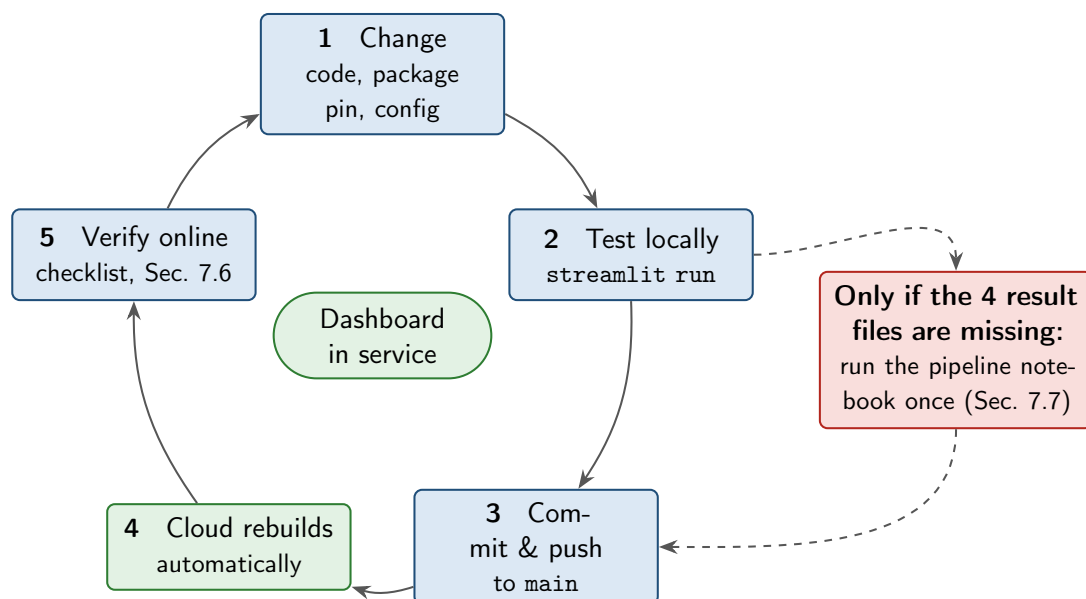


Figure 7.1: Maintenance and redeployment cycle

1. **Change** the file in a local copy of the repository.
2. **Test locally** with `streamlit run Code/app/streamlit_app.py` (Section 2.2) and do the verification in Section 7.6.
3. **Commit and push** to the main branch:

```
git add <changed files>
git commit -m "Short description of the change"
git push
```

4. **Wait for the rebuild.** Streamlit Community Cloud notices the push and restarts the app within a few minutes; if packages changed, it installs them again first.
5. **Verify online** with the same checklist.

7.2 Routine Tasks

Table 7.1: Routine maintenance tasks

When	Task	Reference
Before a presentation or an exam	Open the online app some minutes in advance so that it is awake; run the verification checklist	Sections 2.1.3, 7.6
Monthly	Open the app once; check the Streamlit Community Cloud workspace for warnings or e-mails	Section 7.3
Every 6 months	Check whether the pinned package versions still install; update if needed	Section 7.4
After every change	Complete maintenance cycle	Section 7.1
If files are lost	Regenerate the result files	Section 7.7

7.3 Managing the Online App

The online app is managed in the Streamlit Community Cloud workspace at <https://share.streamlit.io>, after signing in with the GitHub account that owns the repository [Sno26a]. The workspace lists the app with its status. The menu next to it, and the **Manage app** button at the bottom right of the running app (visible only to the signed-in owner), offer:

Table 7.2: Actions in the Streamlit Community Cloud workspace

Action	Use
Logs	Show the app's console output, e.g. package installation errors or Python error messages
Reboot app	Restart the app without changing anything – first aid for a hanging app
Settings	Change the app's address and sharing settings
Delete	Remove the app (the repository is not affected)

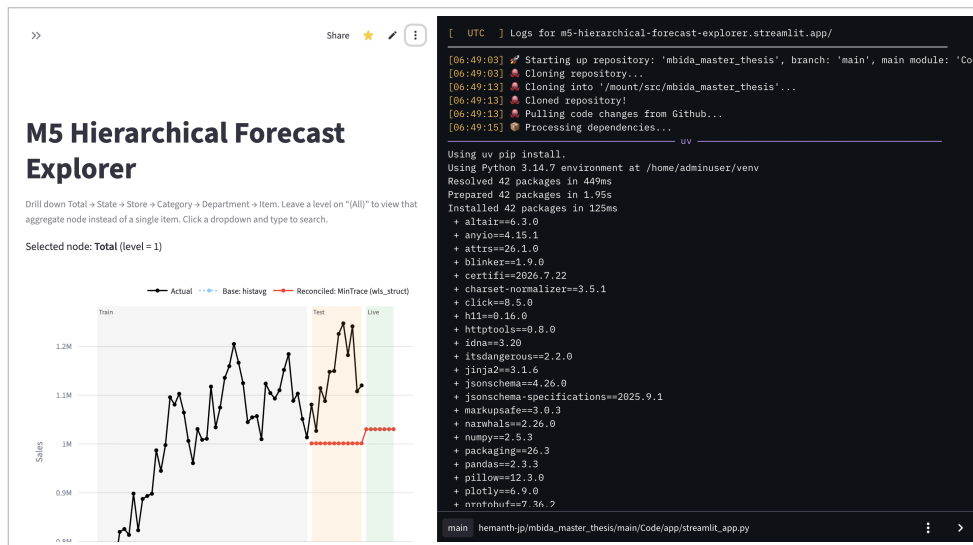


Figure 7.2: The Manage app panel with logs and app menu

The deployment itself is configured as follows. These values are set once when the app is created; they only need to be checked if the app is ever deployed anew.

Table 7.3: Deployment settings of the online app

Setting	Value
Repository	Hemanth-Jp/MBIDA_Master_Thesis
Branch	main
Main file path	Code/app/streamlit_app.py
App Uniform Resource Locator (URL)	m5-hierarchical-forecast-explorer
Python version	3.10 or newer (chosen under Advanced settings)
Package list	Code/app/requirements.txt (found automatically next to the main file)

Caution

The Python version can only be chosen when the app is created. To change it, delete the app and deploy it again with the same settings and the same Uniform Resource Locator (URL).

7.4 Updating Packages

The five packages are pinned to exact versions in `Code/app/requirements.txt`. Pinning guarantees that the online app and every local installation behave identically. To update:

1. In a local copy, change one version number at a time, e.g. `streamlit==1.61.1` to a newer release.

7 Maintenance

2. Reinstall: `pip install -r Code/app/requirements.txt`.
3. Start the app and run the verification checklist (Section 7.6), including light and dark mode.
4. If everything works, complete the maintenance cycle; otherwise return to the previous version number.

Caution

Keep the package name `st-theme` exactly as written. The similarly named `streamlit-theme` is a different package, and the dashboard fails to start with it.

7.5 Changing Settings

Names shown in the sidebar – the list of base models and the reconciliation methods with their labels – come from the settings file `Code/config.py`, which the dashboard shares with the forecasting pipeline. Two rules apply:

- Visible *labels* of reconciliation methods (for example `MinTrace (wls_struct)`) may be renamed freely.
- The technical *names* behind them must match the column names in the result files exactly. If they do not, the dashboard reports “No column found” (Section 8.2.3). They only need to change if the pipeline itself changes.

The look of the Streamlit page (colours, fonts) is set in `.streamlit/config.toml` in the repository root.

7.6 Verification Checklist

Run this list after every change, locally before pushing and online after the rebuild.

- ☐ The page loads without a red error box.
- ☐ The node line shows `Total (level = 1)` at start.
- ☐ The chart shows the actual line, a dotted base line and a solid `MinTrace` line, and the three shaded regions.
- ☐ Drilling down to `CA → CA_1 → FOODS → FOODS_3 → FOODS_3_090` works; the level increases to 6.
- ☐ Each of the three base models changes the dotted line.
- ☐ All three reconciliation methods can be shown together; no “No column found” text appears.
- ☐ Light and dark mode both give a readable chart.
- ☐ The camera button saves a PNG.

7.7 Regenerating the Result Files

This is needed only if the four files in `Code/artifacts/dashboard_data/` are missing or damaged, which the dashboard reports with a red error box (Section 8.2.1). First try to restore them from Git, which is much faster:

```
git checkout -- Code/artifacts/dashboard_data/
```

If that is not possible, recreate them with the thesis pipeline. Because the complete daily data set is processed, this can take half an hour or more and needs a computer with plenty of memory (16 GB of Random Access Memory (RAM) recommended).

1. Download `sales_train_validation.csv` and `calendar.csv` from the M5 competition page [Kag20] into `Code/data/m5/` (see `Code/data/README.md`).
2. Install the pipeline packages in a separate environment:

```
python3 -m venv .venv-pipeline
source .venv-pipeline/bin/activate # Windows: .venv-pipeline\Scripts\
Activate.ps1
pip install -r Code/requirements-pipeline.txt
```

3. Open `Code/notebooks/02_pipeline.ipynb` in Jupyter and run *all cells from top to bottom* (**Run** → **Run All Cells**). The last cell writes the four files.
4. Check that they exist, then commit and push them (Section 7.1):

```
ls Code/artifacts/dashboard_data
git add Code/artifacts/dashboard_data
git commit -m "Regenerate dashboard result files"
git push
```

Note

With unchanged input data and settings, the pipeline reproduces the same results. The online app only shows new files after they have been pushed – running the pipeline locally alone does not change the online dashboard.

7.8 Backup and Rollback

The Git history is the backup: every version of the program and of the result files is stored in the repository. To undo a change that broke the online app:

```
git log --oneline # find the faulty commit
git revert <commit-id> # create a commit that undoes it
git push # the online app rebuilds with the old state
```

7.9 Resetting a Local Installation

If a local installation behaves strangely, rebuilding the virtual environment is quick and safe; it does not touch the repository files:

```
deactivate # if an environment is active
rm -rf .venv # Windows: Remove-Item -Recurse -Force .venv
python3 -m venv .venv
source .venv/bin/activate
pip install -r Code/app/requirements.txt
```

8 Troubleshooting

This chapter is organised by what you see. Find the symptom in Figure 8.1 and go to the section it names. Each section states the symptom, the cause and the solution. Solutions marked (*operator*) need access to the repository or the Streamlit Community Cloud workspace.

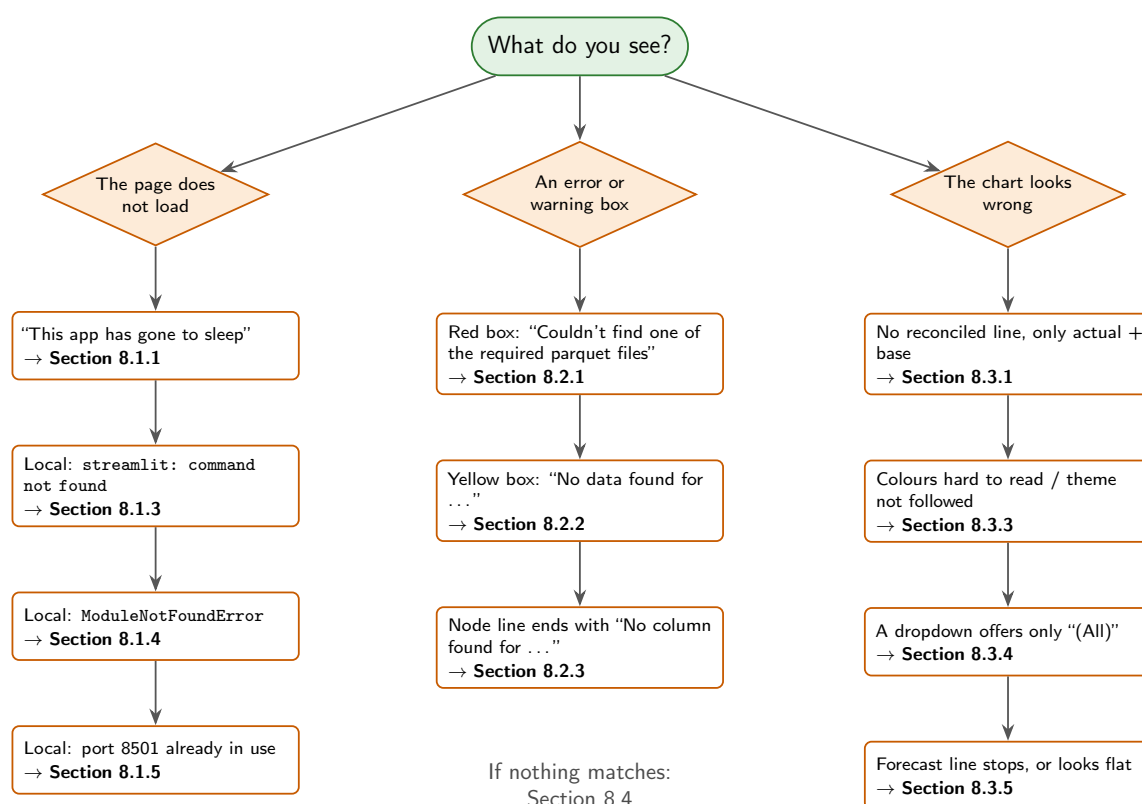


Figure 8.1: Finding the right section from the symptom

8.1 The Page Does Not Load

8.1.1 “This app has gone to sleep”

Symptom

Instead of the dashboard, a page says the app has gone to sleep due to inactivity.

Cause

Streamlit Community Cloud stops apps that have not been used for a while. This

is normal behaviour of the free hosting.

Solution

Click **Yes, get this app back up!** and wait up to a minute (Section 2.1.3). If the app does not start within five minutes: (*operator*) open the logs and reboot the app (Section 7.3).

8.1.2 The Page Stays Blank or Shows “Oh no”

Symptom

The page remains white, keeps loading, or Streamlit shows “Oh no. Error running app.”

Cause

A slow or interrupted connection, a browser extension blocking scripts, or an error during the app’s (re)start.

Solution

Reload the page (**F5**); try a private browser window, which disables most extensions; try another browser. If the error persists for all users: (*operator*) check the logs – a failed package installation after a change is the most frequent reason – and roll back the last change if needed (Section 7.8).

8.1.3 streamlit: command not found

Symptom

Local start fails with `command not found` (macOS/Linux) or “is not recognized as the name of a cmdlet” (Windows).

Cause

The virtual environment is not activated, or the packages were not installed into it.

Solution

Activate the environment (the prompt must start with `(.venv)`), then install and start again:

```
source .venv/bin/activate # Windows: .venv\Scripts\Activate.ps1
pip install -r Code/app/requirements.txt
python -m streamlit run Code/app/streamlit_app.py
```

On Windows, if activation is refused because of the execution policy, run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

8.1.4 ModuleNotFoundError

Symptom

The browser or terminal shows `ModuleNotFoundError: No module named '...'`.

Cause and solution

Depends on the module name:

- `streamlit_theme` – the wrong package `streamlit-theme` was installed instead of `st-theme`. Fix: `pip uninstall streamlit-theme` and `pip install st-theme==1.2.3`.
- `config` – the file `Code/config.py` is missing or the app file was moved out of `Code/app/`. Fix: restore the original folder layout (`git status` shows what changed).
- `pyarrow` or any other package – the requirements were not installed completely. Fix: `pip install -r Code/app/requirements.txt`.

8.1.5 Port 8501 Already in Use

Symptom

Streamlit reports that port 8501 is not available, or it starts on 8502 instead.

Cause

Another Streamlit app – often an earlier start of this dashboard – is still running.

Solution

Stop the other app with `Ctrl+C` in its terminal, or start this one on another port:

```
streamlit run Code/app/streamlit_app.py --server.port 8600
```

8.2 Error and Warning Messages

8.2.1 “Couldn’t find one of the required parquet files”

Symptom

A red box with this text and the name of the missing file; no chart (Figure 8.2).

Cause

One of the four result files is missing from `Code/artifacts/dashboard_data/`, e.g. because it was deleted, not committed or not downloaded.

Solution

(operator) Restore the files from Git or regenerate them (Section 7.7).

8 Troubleshooting



Figure 8.2: Error message when a result file is missing

8.2.2 “No data found for ...”

Symptom

A yellow box instead of the chart, naming a node such as `Total/CA/CA_1/...`

Cause

The dropdown list and the sales file do not match – normally only possible if the result files come from different pipeline runs.

Solution

Choose another node to continue. (*operator*) Make sure all four files were written by the same pipeline run; restore them from Git or regenerate them together (Section 7.7).

8.2.3 “No column found for ...”

Symptom

The node line ends with `No column found for: MinTrace (wls_struct)` (or another method), and that method’s line is missing.

Cause

The technical name of the method in `Code/config.py` does not match the result files – usually after the pipeline or the settings were changed.

Solution

(*operator*) Align the method names in `config.py` with the result files (Section 7.5), or undo the last change.

8.3 The Chart Looks Wrong

8.3.1 No Reconciled Line

Symptom

Only the actual and the dotted base line are visible.

Cause

The **Reconciliation method(s)** box is empty, or the line was hidden by clicking its legend entry.

Solution

Select at least one method in the box; click the greyed-out legend entry to show the line again.

8.3.2 A Reconciled Line Hides Another Line

Symptom

Fewer lines are visible than the legend lists.

Cause

Two forecasts are identical and lie exactly on top of each other – for example Top-Down and the base forecast at the Total, or Bottom-Up and the base forecast at item level (Section 5.5).

Solution

None needed. Hide one line via the legend to check.

8.3.3 Colours Hard to Read

Symptom

Lines or labels have too little contrast, e.g. after switching between light and dark mode.

Cause

The chart detects the theme when it is drawn; in rare cases the first drawing happens before the theme is known.

Solution

Change any control once or reload the page (**F5**); the chart is then drawn with the right colours. Choose **Light** in the page menu for printing.

8.3.4 A Dropdown Offers Only “(All)”

Symptom

For example, **Department** cannot be opened to any value.

Cause

A dropdown above it is still on (All). This is intended (Section 5.2).

Solution

Choose values in all dropdowns above it first, from top to bottom.

8.3.5 Forecast Looks Flat, or Is Zero

Symptom

The forecast is a horizontal line, identical for all three base models, or lies at zero.

Cause

`histavg` is flat by definition. For items that sell rarely, `ets` and `theta` fall back to `histavg`, and forecasts for items that hardly sell at all are close to zero (Section 5.4).

Solution

None needed; this is correct behaviour. Compare with the department or store above the item to see the models' differences.

8.4 General Procedure

If none of the symptoms above matches, work through these steps in order:

1. Reload the page (**F5**); try a private window and a second browser.
2. Check whether the problem also occurs online *and* locally. If only online: (*operator*) check the logs and reboot (Section 7.3). If only locally: reset the local installation (Section 7.9).
3. Compare with the verification checklist (Section 7.6) to narrow down which function fails.
4. Report the problem as described in Section 9.4.

9 Help

This chapter collects quick answers: frequently asked questions, a glossary, a one-page quick reference, and how to get further help.

9.1 Frequently Asked Questions

Do I need an account or a password? No. Anybody with the link can use the dashboard.

Can I use my own sales data? No. The dashboard shows the pre-computed thesis results for the M5 data only (Section 6.9).

Why does the black line stop at the end of 2015? From January 2016 onwards the chart is in the live window, which is kept forecast-only. The data set does contain real sales up to April 2016, but they are not drawn, so the live window always shows forecasts without a comparison line. From May 2016 there is no real sales data at all (Section 6.5).

Why are some lines identical? Some methods leave certain levels unchanged by design: Top-Down keeps the total, Bottom-Up keeps the items. At those levels the reconciled line lies exactly on the base line (Section 5.5).

Which forecast is the best? It depends on the level and the node. Averaged over the test year, Bottom-Up with `theta` or `ets` had the lowest error on most levels (Table 6.6). The dashboard lets you check this for a single node.

Why are forecasts for single items so poor? Single products in single stores often sell only a few pieces per month, with many months at zero. Such irregular sales are hard to predict for any method; the errors at item level are around 36–42 % compared with 4–7 % at the total.

Can I compare two stores in one chart? No, the chart shows one node at a time. Save both charts as images (camera button) or open the dashboard in two browser tabs side by side.

Can I download the numbers? Not from the dashboard. Hovering shows exact values; the complete result files are in the repository (`Code/artifacts/dashboard_data/`).

Does the dashboard work on a phone? Yes. The sidebar is hidden at first; open it with the arrow at the top left. A larger screen is more comfortable.

The app says it is asleep – is something broken? No. Click the button to wake it up and wait up to a minute (Section 2.1.3).

Do my selections affect other users? No. Each browser tab has its own selections; nothing is saved or shared.

9.2 Glossary

Actual Units really sold in a month.

Aggregate node A node above item level, i.e. a sum of several items.

Base forecast Forecast of a node made independently of all other nodes.

Bottom-Up Reconciliation that adds up item forecasts.

Coherent Forecasts are coherent when the parts add up to the whole on every level.

Exponential smoothing (ETS) Forecasting model that weights recent months more strongly and models trend and season.

Hierarchy The Total–State–Store–Category–Department–Item structure of the sales.

Historical average Forecasting model that predicts the average of all past months.

Level Depth of a node in the hierarchy, from 1 (Total) to 6 (Item).

Live Forecast-only window, January–July 2016.

MinTrace Reconciliation that adjusts all levels at once, as little as possible.

Node One point of the hierarchy; the chart always shows one node.

Parquet Compact file format in which the results are stored.

Reconciliation Adjusting base forecasts so that they become coherent.

sMAPE Average relative forecast error in percent; lower is better.

Streamlit Python framework in which the dashboard is written; also the hosting service.

Test Comparison window, the year 2015.

Theta Forecasting model combining a long-term trend line with a short-term smoothed view.

Top-Down Reconciliation that splits the total forecast by historical shares.

Train Learning window, February 2011 – December 2014.

9.3 Quick Reference

M5 Hierarchical Forecast Explorer – Quick Reference	
Open: https://m5-hierarchical-forecast-explorer.streamlit.app/ Local: <code>streamlit run Code/app/streamlit_app.py</code>	
I want to ...	Do this
see the grand total	State = (All)
see one state / store	choose State , then Store
see one item	choose all five dropdowns; type the item number to search
go up one level	set the lowest chosen dropdown to (All)
change the dotted line	Base model: <code>histavg</code> / <code>ets</code> / <code>theta</code>
compare reconcilers	add methods in Reconciliation method(s) ; × removes
read a value	hover over the line
zoom / reset	drag a rectangle / double-click
hide a line	click its legend entry
save the chart	camera button (top right of chart)
switch light / dark	page menu ... → Settings
wake a sleeping app	Yes, get this app back up!
Chart: black = actual · dotted = base · solid coloured = reconciled · Train 2011–14 Test 2015 Live 2016	

9.4 Reporting a Problem

If a problem cannot be solved with Chapter 8, report it as an issue in the GitHub repository (https://github.com/Hemanth-Jp/MBIDA_Master_Thesis, tab **Issues**) or directly to the author. Please include:

- online or local use; for local use the operating system and Python version,
- the selected node (copy the node line) and the model and method settings,
- the exact message text, or a screenshot,
- what you did just before the problem occurred.

9.5 Further Information

- The master's thesis *Hierarchical Time Series Forecasting in Retail Demand* describes the data preparation, the models, the reconciliation methods and the results in full.
- *Forecasting: Principles and Practice* [HA21] is a free, readable introduction to forecasting and to hierarchical reconciliation.
- The M5 competition and its data are described in [MSA22].
- Streamlit and Streamlit Community Cloud documentation: [Sno26b; Sno26a].

Bibliography

- [HA21] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: <https://otexts.com/fpp3/>.
- [Kag20] Kaggle, *M5 forecasting – accuracy*, Online competition, 2020. Accessed: Sep. 25, 2026. [Online]. Available: <https://www.kaggle.com/competitions/m5-forecasting-accuracy>.
- [MSA22] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “M5 accuracy competition: Results, findings, and conclusions,” *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022. DOI: 10.1016/j.ijforecast.2021.11.013.
- [OGL+22] K. G. Olivares et al., *HierarchicalForecast: A reference framework for hierarchical forecasting in python*, arXiv preprint arXiv:2207.03517, 2022. [Online]. Available: <https://arxiv.org/abs/2207.03517>.
- [Plo26] Plotly Technologies Inc., *Plotly python graphing library*, Online, 2026. Accessed: Sep. 25, 2026. [Online]. Available: <https://plotly.com/python/>.
- [Sno26a] Snowflake Inc., *Streamlit community cloud documentation*, Online, 2026. Accessed: Sep. 25, 2026. [Online]. Available: <https://docs.streamlit.io/deploy/streamlit-community-cloud>.
- [Sno26b] Snowflake Inc., *Streamlit documentation*, Online, 2026. Accessed: Sep. 25, 2026. [Online]. Available: <https://docs.streamlit.io>.
- [WAH19] S. L. Wickramasuriya, G. Athanasopoulos, and R. J. Hyndman, “Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization,” *Journal of the American Statistical Association*, vol. 114, no. 526, pp. 804–818, 2019. DOI: 10.1080/01621459.2018.1448825.